



8INF257

Informatique Mobile

Hiver 2025

© Eduardo Mendes

8INF257: Informatique Mobile

© Eduardo Mendes

Tests

Pourquoi tester ?



Pourquoi tester?

Un logiciel sans tests



comportement imprévisible

Les tests permettent de

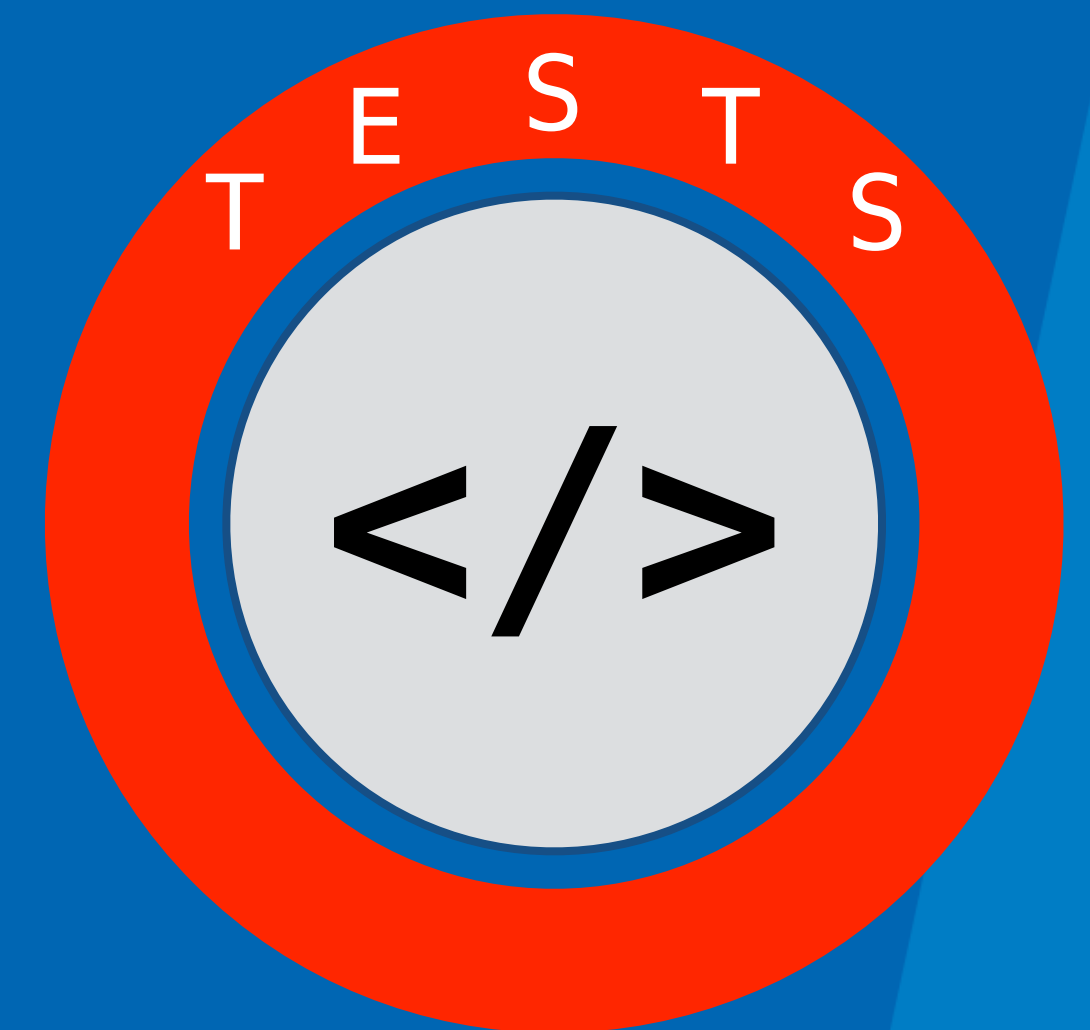
**vérifier que
le code fonctionne
comme prévu**

**détecter
les bugs tôt**

**éviter
les régressions**

Tests | Un processus constant

- Le test des logiciels est **un processus constant** à exécuter **un programme avec des données simulant** les entrées des utilisateurs
- L'objectif est **d'observer le comportement du programme** pour vérifier s'il réalise les tâches prévues
- **Les tests réussissent** si le comportement est conforme aux attentes
- **Les tests échouent** si le comportement diffère des attentes



A silhouette of a person sitting in a meditative pose on a rocky mountain peak. The background is a warm, orange-hued sunset or sunrise sky with soft clouds. The person's hands are resting on their knees in a mudra.

Les tests

*Si le programme fonctionne correctement
pour un ensemble d'entrées spécifiques,
cela suggère qu'il peut fonctionner correctement
pour un ensemble plus large d'entrées similaires*

Tests

- **Objectif des tests**

- Convaincre les développeurs et chefs de produit que le logiciel est prêt pour la distribution
- Identifier les bugs avant la mise en production

- **Limites des tests**

- Impossible de garantir qu'un programme est sans défauts
- Certaines entrées ou combinaisons d'entrées non testées peuvent provoquer des échecs

**Les tests sont
souvent négligés**



Pourquoi les tests sont-ils négligés ?

**Pression
de livraison**

**Manque
de temps**

**« Ça fonctionne
déjà »**

Pourquoi les tests sont-ils négligés ?

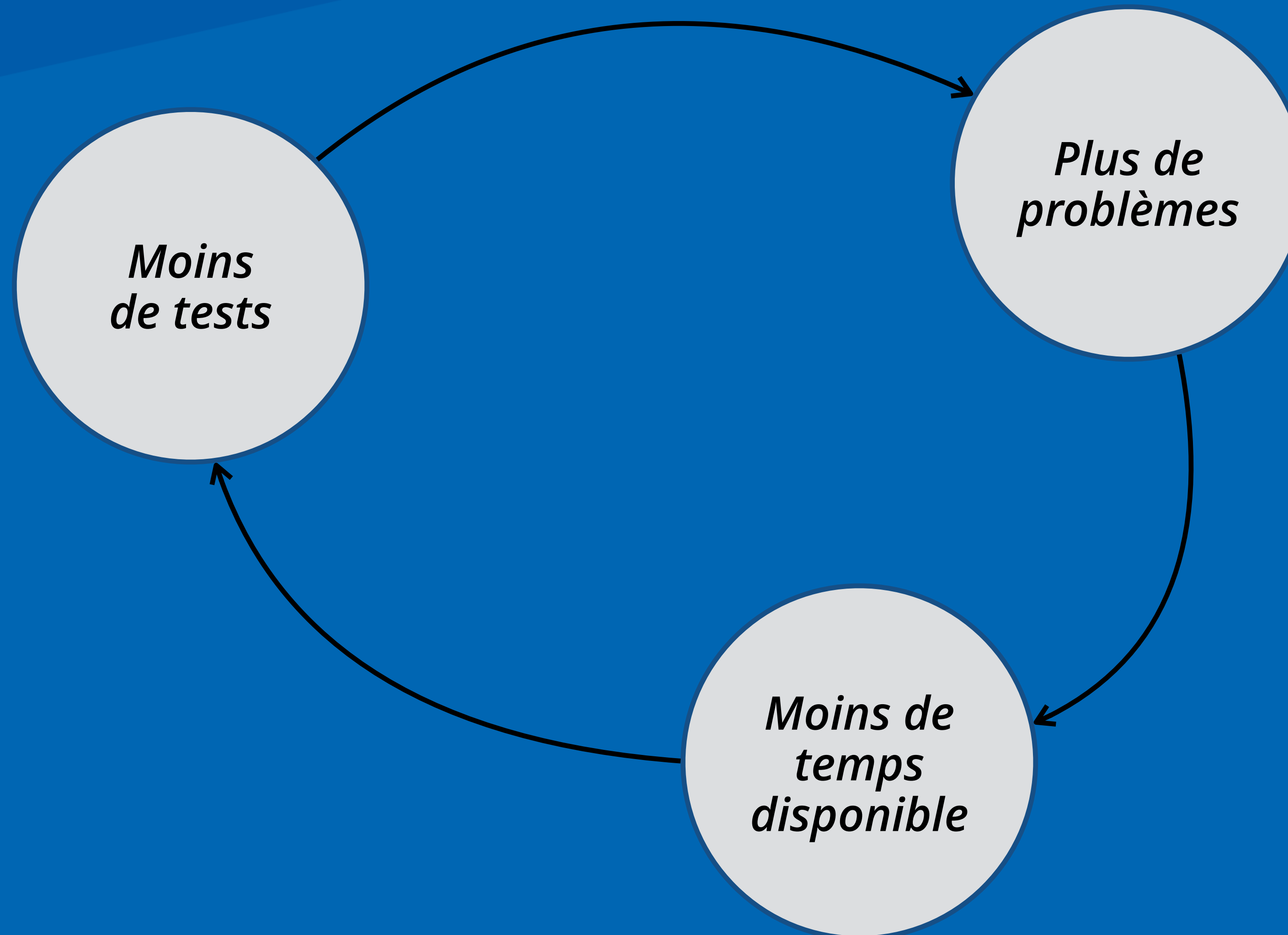
**Pression
de livraison**

**Manque
de temps**

**« Ça fonctionne
déjà »**

Conséquence : accumulation de bugs

Spirale de la mort





Types de tests

Tests

- Les tests ne sont pas seulement nécessaires, mais **vitaux** pour nos applications
- Une application bien testée est plus robuste et fiable
- Les tests augmentent la confiance dans le code

Tests

- Les tests ne sont pas seulement nécessaires, mais **vitaux** pour nos applications
- Une application bien testée est plus robuste et fiable
- Les tests augmentent la confiance dans le code

Tests unitaires

Tester la logique métier

Tests d'interface

Tests de composants individuels

Tests d'intégration

Tests d'interactions entre composants



Test unitaire

Test unitaire

- Teste **une petite partie isolée** du code
- **Focus**
 - Vérifient le comportement précis des objets ou des méthodes individuelles
- **Automatisation**
 - Les tests doivent être automatisés pour une efficacité accrue

Test unitaire | Les unités de code

- **Une unité** de code est un élément ayant **une responsabilité clairement définie**
- **Quelles sont les parties isolées à tester ?**
 - Les **fonctions** ou **méthodes** individuelles au sein d'un objet
 - Les **classes d'objets** avec plusieurs attributs et méthodes
 - Les **composants complexes** avec des interfaces bien définies pour accéder à leur fonctionnalité
- Lors du développement d'une unité de code, il est essentiel **de développer simultanément des tests** pour cette unité

Test unitaire

Les tests unitaires reposent sur un **principe général simple :**

Si une unité de programme se comporte comme prévu pour un ensemble d'entrées ayant des caractéristiques communes, elle se comportera de la même manière pour un ensemble plus large partageant ces mêmes caractéristiques.



Anatomie d'un test

Arrange

(Préparation)

- Créer objets
- Préparer données
- Configurer mocks
- Initialiser environnement

ACT

(Action)

- Appeler la méthode testée
- Exécuter l'opération
- Déclencher l'action

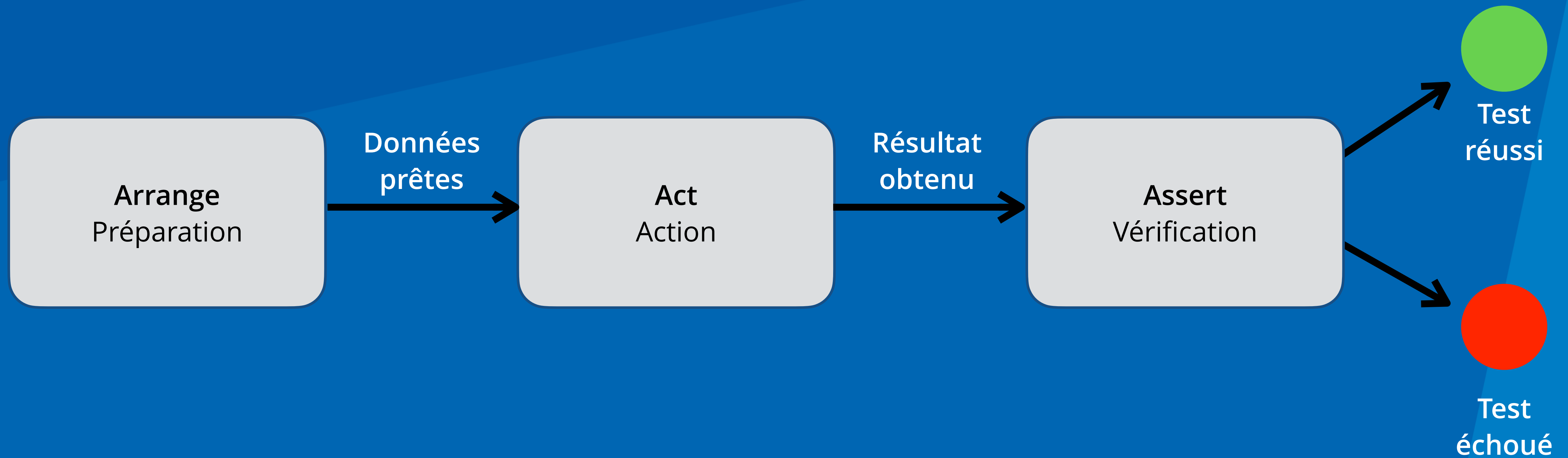
ASSERT

(Vérification)

- Vérifier le résultat
- Comparer avec valeur attendue
- Vérifier les effets

- **Préparation (Arrange)** : Configuration des données et objets nécessaires
- **Action (Act)** : Exécution de la fonction/méthode à tester
- **Vérification (Assert)** : Vérification des résultats attendus

Anatomie d'un test



- **Préparation (Arrange)** : Configuration des données et objets nécessaires
- **Action (Act)** : Exécution de la fonction/méthode à tester
- **Vérification (Assert)** : Vérification des résultats attendus

Flux d'exécution d'un test

- **Préparation** : On initialise les objets, données, et états nécessaires

```
val database = FakeDatabase()  
val useCase = UpsertStoryUseCase(database)  
val validStory = Story(1, "Titre", "Auteur")
```

- **Action** : On exécute l'opération à tester

```
// Appel de la fonction testée  
useCase.invoke(validStory)
```

- **Vérification** : On confirme que le résultat est celui attendu

```
assertTrue(database.stories.contains(validStory))  
assertEquals(1, database.stories.size)
```

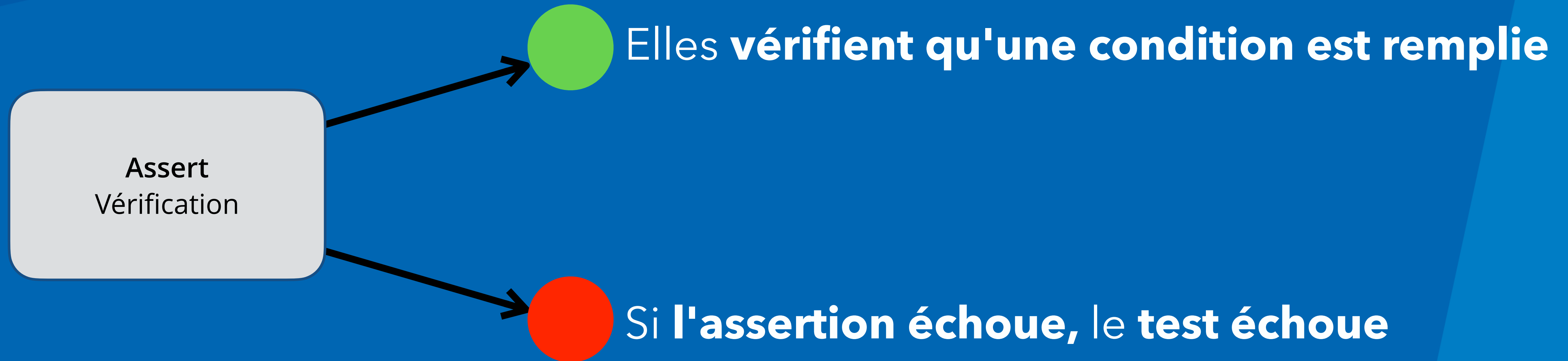

Vérification et assertions

```
assertTrue(database.stories.contains(validStory))  
assertEquals(1, database.stories.size)
```

- Les assertions déterminent si le test réussit ou échoue
- Sans assertion, un test est inutile

Le rôle des assertions

- Les **assertions** sont le **cœur** de la vérification dans les tests



Le rôle des assertions

- Pour vérifier l'égalité
 - *assertEquals(expected, actual)*
- Pour vérifier qu'une **condition est vraie**
 - *assertTrue(condition)*
- Pour vérifier qu'une **condition est fausse**
 - *assertFalse(condition)*
- Pour vérifier qu'une **valeur n'est pas nulle**
 - *assertNotNull(value)*
- Pour vérifier qu'une exception est lancée
 - *assertThrows(exception)*



Comment choisir les données de test ?

On ne teste pas toutes les valeurs possibles

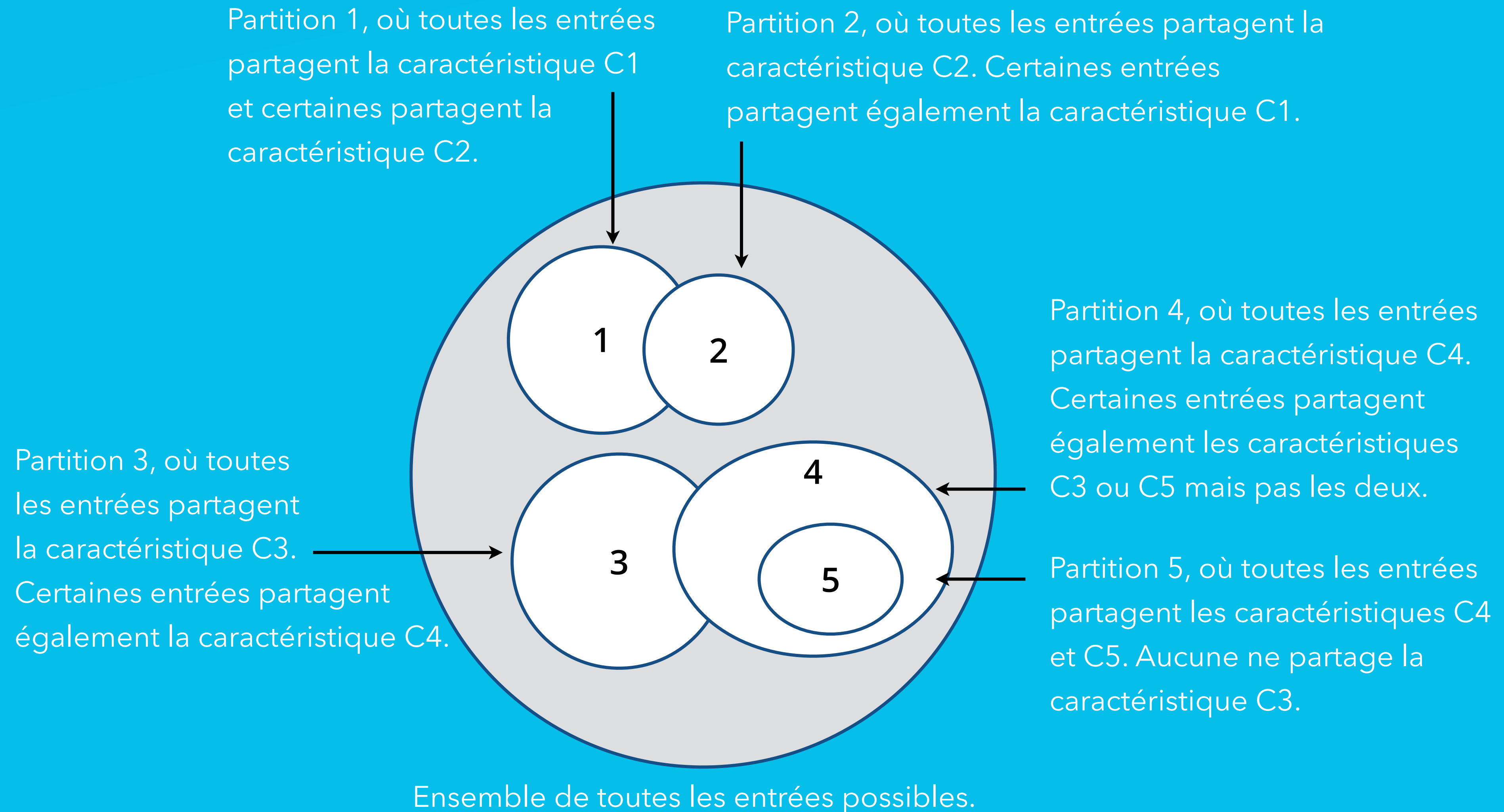
On regroupe les entrées similaires



Partitions d'équivalence

- Pour tester efficacement un programme,
il est nécessaire **d'identifier des ensembles d'entrées**
qui seront traités de manière identique dans le code
- Les **partitions d'équivalence** doivent inclure
non seulement les entrées
produisant des valeurs correctes,
mais aussi des partitions d'erreurs,
où les entrées sont délibérément incorrectes

Partitions d'équivalence



Exemple

- Imaginez que vous testez **un formulaire en ligne**.
- **Les caractéristiques** pourraient être différentes combinaisons comme :
 - *Le champ "nom" est rempli ou non (C1).*
 - *L'e-mail est au bon format ou non (C2).*
 - *Le mot de passe est suffisamment fort ou non (C3), et ainsi de suite.*
- Chaque partition correspondrait à un groupe d'entrées partageant des combinaisons de ces caractéristiques.

Exemple avec code

```
fun name_check(name):  
    """  
    Valide un nom en fonction de règles spécifiques :  
    - Doit contenir entre 2 et 40 caractères.  
    - Ne peut pas commencer par un trait d'union (-) ou une apostrophe (').  
    - Ne peut contenir que des caractères alphabétiques,  
      des traits d'union et des apostrophes.  
    - Ne peut pas contenir de doubles traits d'union ou de caractères invalides.  
    """  
    if len(name) < 2 or len(name) > 40:  
        return False  
    if name.startswith('-') or name.startswith("'"):  
        return False  
    if re.search(r"^[^\w'-]", name): # Uniquement les lettres, les tirets et les apostrophes  
        return False  
    if '--' in name or "'" in name:  
        return False  
    return True
```


Partitions d'équivalence pour la fonction de vérification des noms

- **Partitions d'équivalence 1 : Noms corrects**

- Les entrées contiennent uniquement des caractères alphabétiques et ont une longueur comprise entre 2 et 40 caractères.

```
fun test_alpha_name(self):  
    assertTrue(name_check('Sommerville'))
```

Partitions d'équivalence pour la fonction de vérification des noms

- **Partitions d'équivalence 2 : Noms corrects**

- Les entrées contiennent uniquement des caractères alphabétiques, des tirets ou des apostrophes, et ont une longueur comprise entre 2 et 40 caractères.

```
fun test_alpha_name(self):  
    assertTrue(name_check("O'Connell"))  
    assertTrue(name_check("Washington-Wilson"))
```


Partitions d'équivalence pour la fonction de vérification des noms

- **Partitions d'équivalence 3 : Noms incorrects**

- Les entrées ont une longueur comprise entre 2 et 40 caractères mais incluent des caractères interdits.

```
fun test_double_quote(self):  
    assertFalse(name_check("This is &maliciouscod%"))
```

Partitions d'équivalence pour la fonction de vérification des noms

- **Partitions d'équivalence 4 : Noms incorrects**

- Les entrées contiennent des caractères autorisés mais sont soit d'un seul caractère, soit dépassent 40 caractères.

```
fun test_name_too_long(self):  
    assertFalse(name_check('Thisisalongstringwithmorethan40charactersfrombeginningtoend'))
```


Partitions d'équivalence pour la fonction de vérification des noms

- **Partitions d'équivalence 5 : Noms incorrects**

- Les entrées ont une longueur correcte mais commencent par un tiret ou une apostrophe.

```
fun test_name_starts_with_hyphen(self):  
    assertFalse(name_check( '-Sommerville' ))
```

```
fun test_name_starts_with_quote(self):  
    assertFalse(name_check( "'Reilly'" ))
```

Partitions d'équivalence pour la fonction de vérification des noms

- **Partitions d'équivalence 6 : Noms incorrects**

- Les entrées incluent des caractères valides et ont une longueur correcte mais contiennent un double tiret, un texte entre guillemets, ou les deux.

```
fun test_double_quote(self):  
    assertFalse(name_check("This is 'maliciouscode'"))  
  
fun test_name_with_double_hyphen(self):  
    assertFalse(name_check('--badcode'))
```


Lignes directrices pour les tests

Limites

Si une partition a des bornes supérieures et inférieures (par exemple, longueur des chaînes, nombres), choisissez des entrées aux extrémités de la plage.

```
fun test_limites():  
    assert verifier_longueur("") == "Erreur : longueur invalide" # Limite inférieure  
    assert verifier_longueur("a") == "Erreur : longueur invalide" # Juste sous la limite  
    assert verifier_longueur("abc") == "Longueur valide" # Limite acceptable  
    assert verifier_longueur("abcdefghij") == "Longueur valide" # Limite supérieure  
    assert verifier_longueur("abcdefghijkl") == "Erreur : longueur invalide" # Au-delà de la limite
```

Lignes directrices pour les tests

Forcer les erreurs

Sélectionnez des entrées qui obligent le système à générer tous les messages d'erreur possibles.

```
fun test_erreurs():  
    assert division(10, 0) == "Erreur : division par zéro" # Division par zéro  
    try:  
        division("10", 2) # Erreur de type  
    except TypeError:  
        pass # Test réussi si TypeError est levé
```


Lignes directrices pour les tests

Remplir les buffers

Choisissez des entrées qui provoquent un débordement de tous les buffers d'entrée.

```
fun test_buffer():  
    assert remplir_buffer("a" * 50) == "Buffer rempli correctement" # Limite exacte  
    assert remplir_buffer("a" * 51) == "Erreur : dépassement du buffer" # Dépassement
```

Lignes directrices pour les tests

Répéter les tests

Répétez les mêmes entrées
ou séries d'entrées plusieurs fois.

```
fun test_repetition():  
    for _ in range(10): # Répéter 10 fois  
        assert addition(5, 3) == 8 # Résultat attendu
```


Lignes directrices pour les tests

Débordement et sous-dépassement

Si votre programme effectue des calculs numériques, testez avec des entrées qui produisent des résultats très grands ou très petits.

```
test_debordement():  
    assert multiplication_grande(1) > 0    # Résultat valide  
    assert multiplication_grande(1e308) == "Erreur : dépassement numérique" # Dépassement
```

Lignes directrices pour les tests

Null et zéro

Testez toujours avec des pointeurs nuls, des chaînes vides et des séquences vides.

Pour les entrées numériques, testez avec zéro.

```
fun test_null_zero():  
    assert traiter_chaine(None) == "Erreur : chaîne vide ou nulle" # Chaîne nulle  
    assert traiter_chaine("") == "Erreur : chaîne vide ou nulle"  # Chaîne vide  
    assert traiter_chaine("Python") == "Chaîne traitée : Python"  # Chaîne valide
```


Lignes directrices pour les tests

Vérifiez les comptes

Pour les listes et leurs transformations, comptez les éléments dans chaque liste et assurez-vous qu'ils sont cohérents après chaque transformation.

```
fun test_verification_comptes():  
    liste = [1, 2, 3]  
    transformee = transformer_liste(liste)  
    assert len(liste) == len(transformee) # Vérifier la cohérence  
    assert transformee == [2, 4, 6] # Vérifier les valeurs transformées
```

Lignes directrices pour les tests

Un seul élément

Si votre programme traite des séquences, testez avec des séquences contenant une seule valeur.

```
fun test_un_seul_element():  
    assert traiter_sequence([5]) == [25] # Une seule valeur  
    assert traiter_sequence([1]) == [1]  # Cas limite
```


JUnit

- **JUnit** est le framework pour appliquer les tests en Java, Kotlin
- **JUnit fournit des annotations**
pour structurer l'implémentation du code de test
- Ces annotations sont utilisées
pour différentes phases du cycle de vie des tests

Annotations JUnit vs Phases de test

Annotation JUnit	Phases AAA	Description
@Before	Préparation : Arrange	- Exécuté avant chaque test - Prépare l'environnement commun à tous les tests
@Test	Exécution du test : Contient Arrange, Act et Assert	Définit une méthode de test qui contient généralement les 3 phases
@After	Nettoie après Assert	- Exécuté après chaque test - Nettoie les ressources

Où tester dans notre architecture ?

La logique métier
se trouve dans Use Cases

Donc

tests unitaires = tests des use cases



Tests unitaires dans Stories

- Tester **la logique métier**
- Doivent être **rapides** pour encourager leur exécution fréquente
- Dans notre application, la logique métier réside dans nos cas d'utilisation (use cases):
 - *DeleteStoryUseCase*
 - *GetStoriesUseCase*
 - *GetStoryUseCase*
 - *UpsertStoryUseCase*

Exemple | UpsertStoryUseCase

- **Insère ou met à jour une histoire** dans la base de données
- Effectue des validations :
 - Vérifie que **le titre n'est pas vide**
 - Vérifie que **la description n'est pas vide**

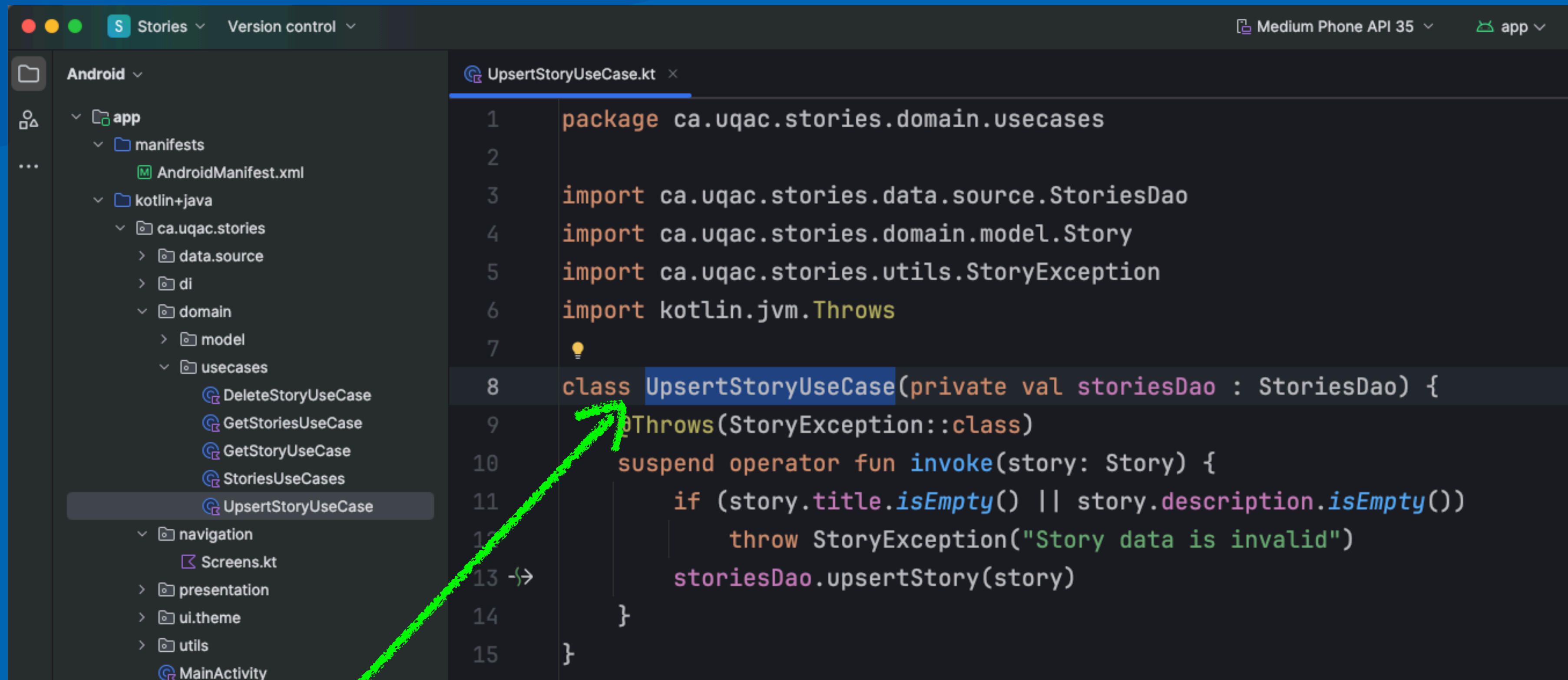
Exemple | UpsertStoryUseCase

- Insère ou met à jour une histoire dans la base de données
- Effectue une validation :
 - Vérifie que le titre n'est pas vide
 - Vérifie que l'auteur n'est pas vide
- ***Comment tester cette logique?***

Création d'un Test Unitaire

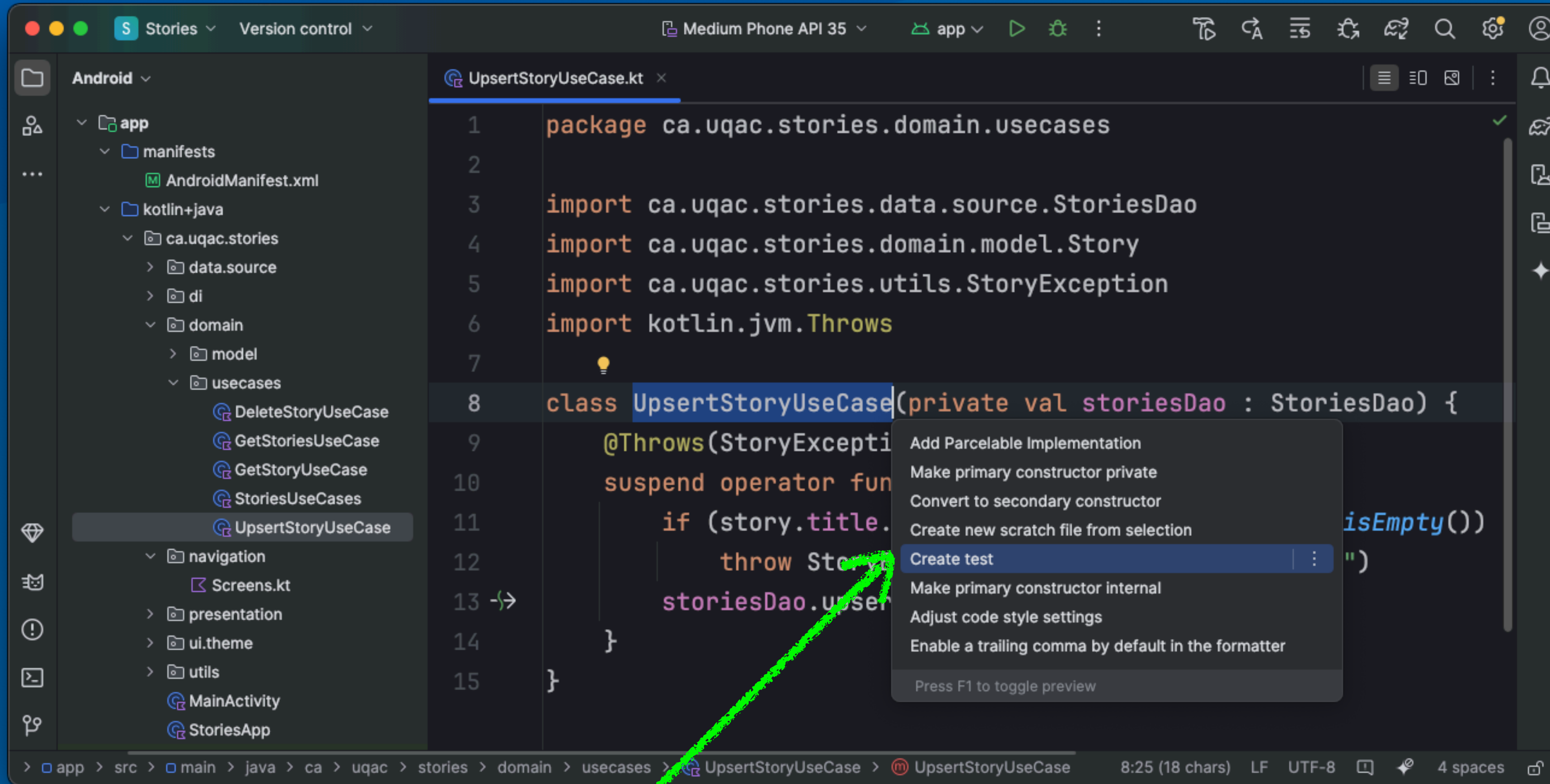
- **Les tests unitaires** sont situés dans **app/src/test**
 - **Créez un test avec Alt+Enter** : Create test
 - Sélectionnez **JUnit4** comme framework (compatibilité Android)
 - Sélectionnez le package **test**
 - Ce package contient des bibliothèques de tests unitaires

Test | Création de tests avec l'IDE



Sélectionnez la classe pour laquelle
le test sera créé et Alt+Enter

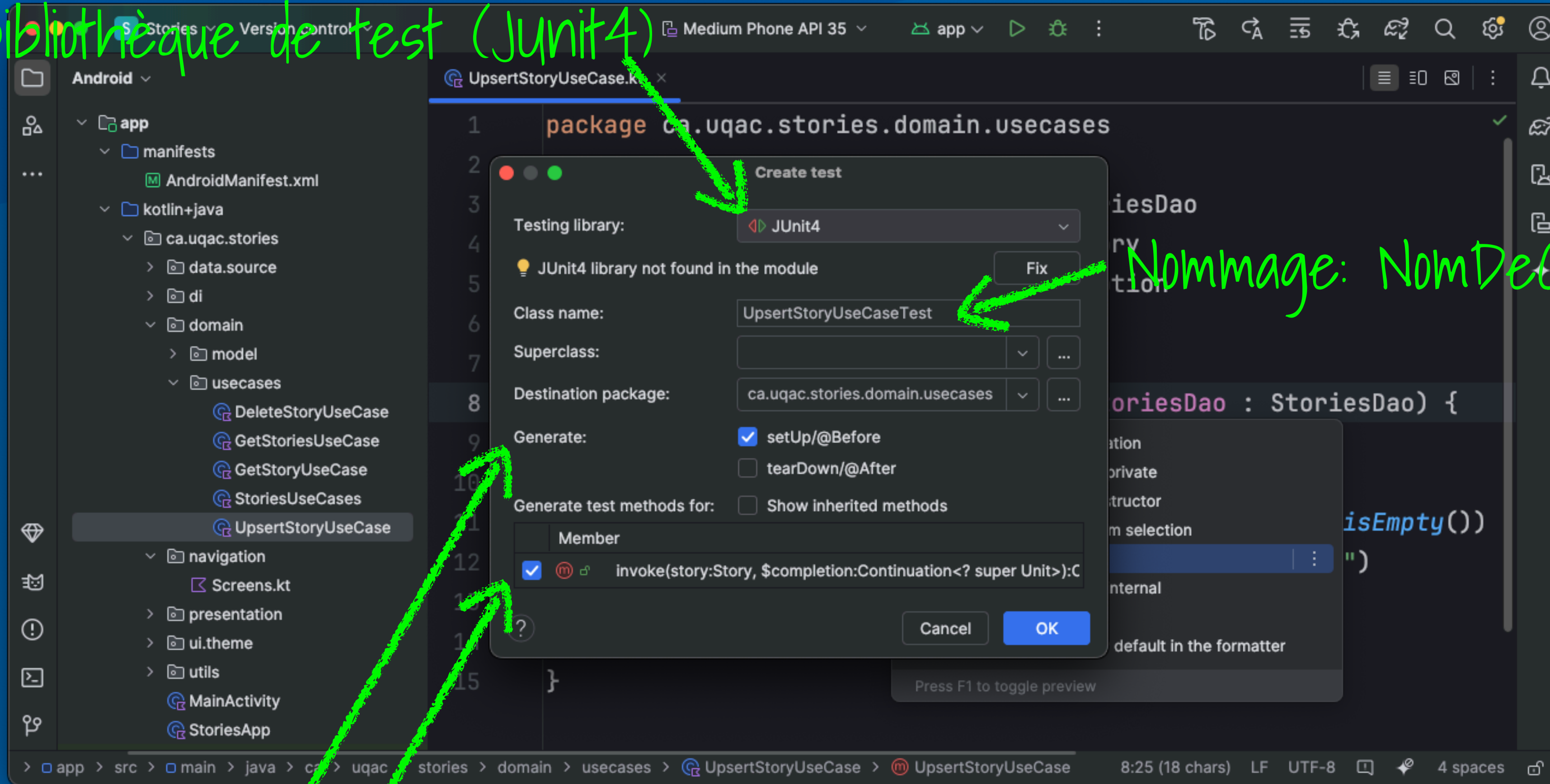
Test



Sélectionnez « Create test »

Test

Spécifier la bibliothèque de test (JUnit4)



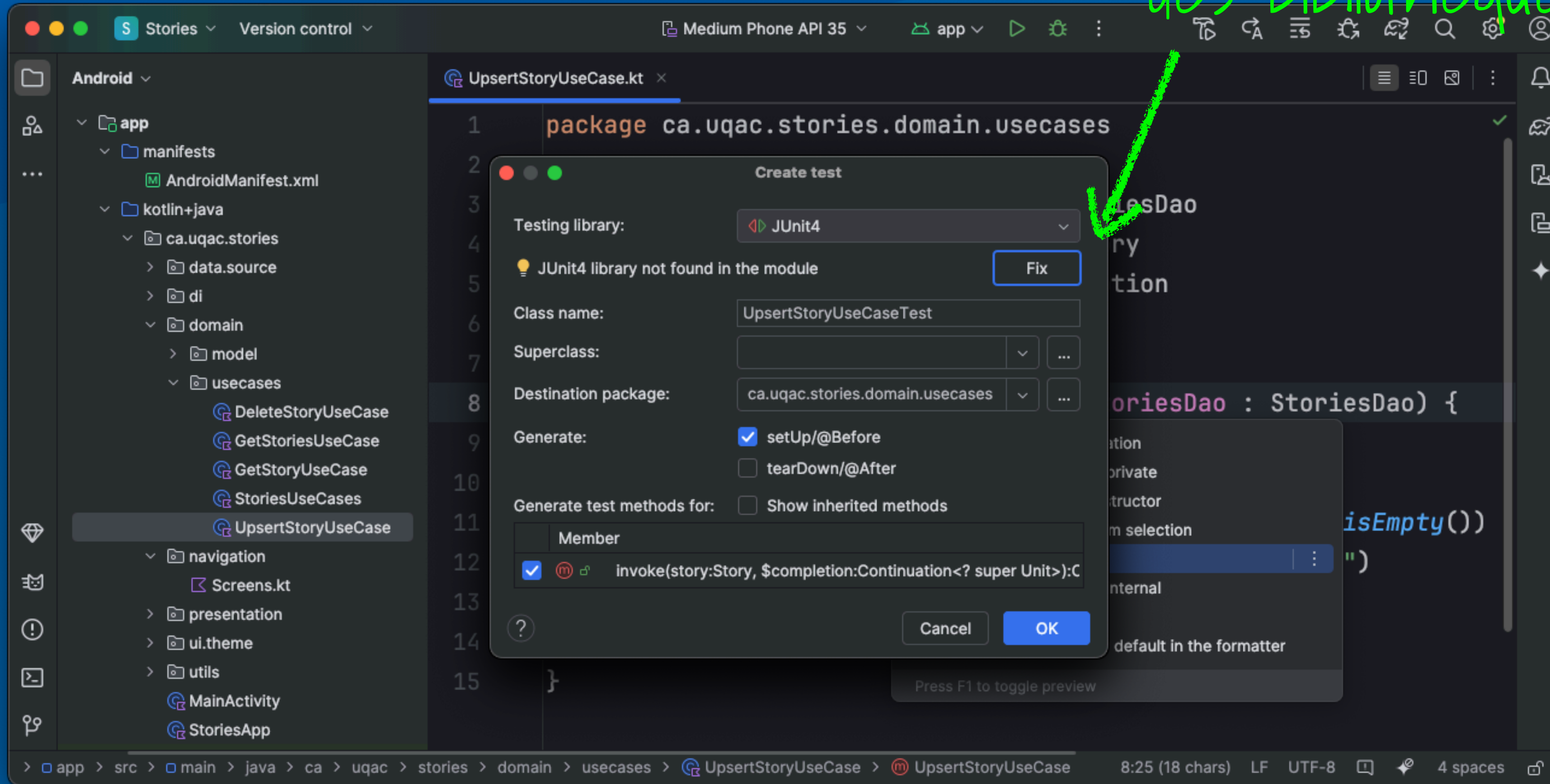
Nommage: NomDeClasse + "Test"

Génération de code:

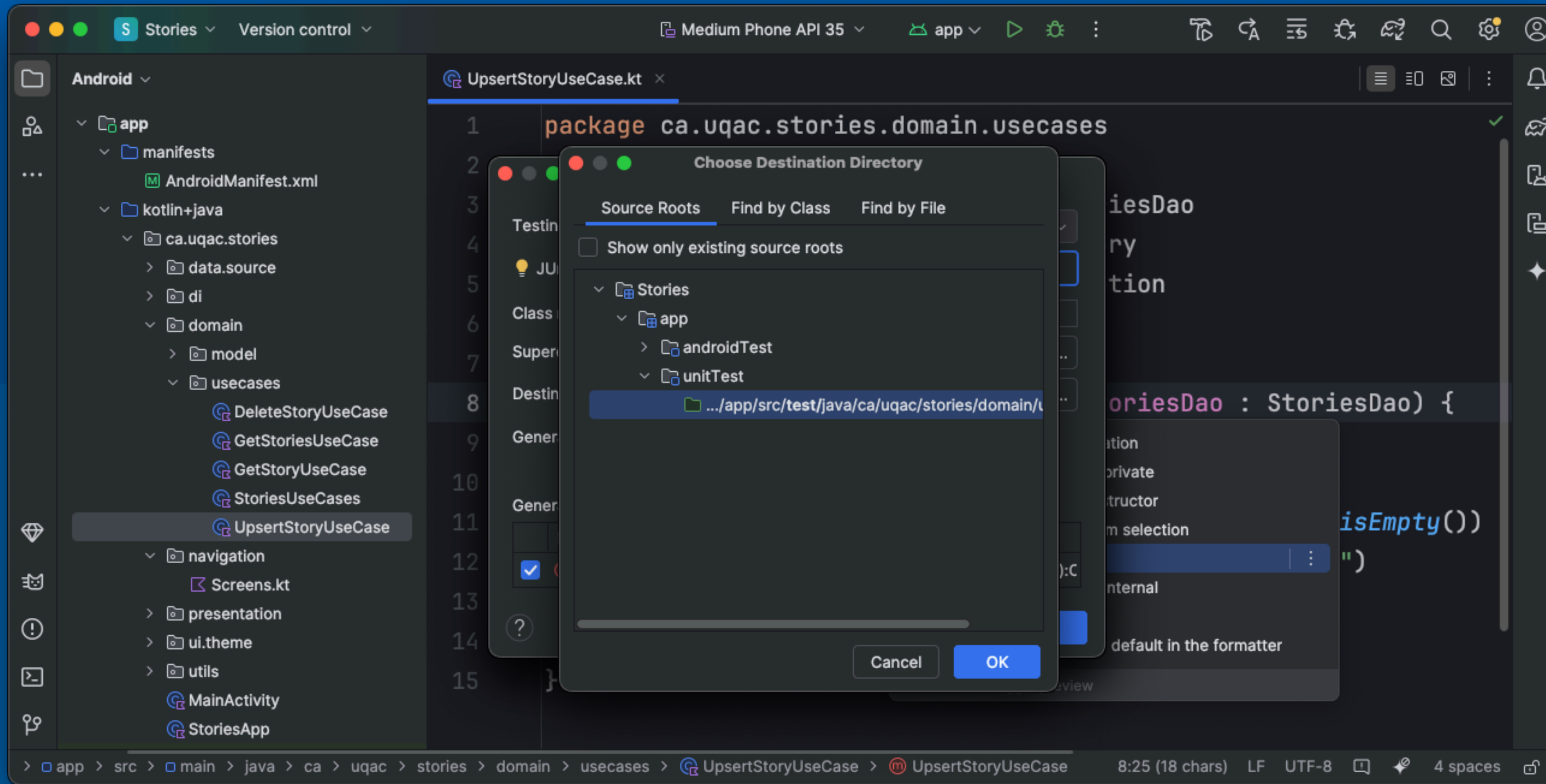
- Fonction setup
- Squelette des tests

Test

< Fix > pour l'installation
des bibliothèques



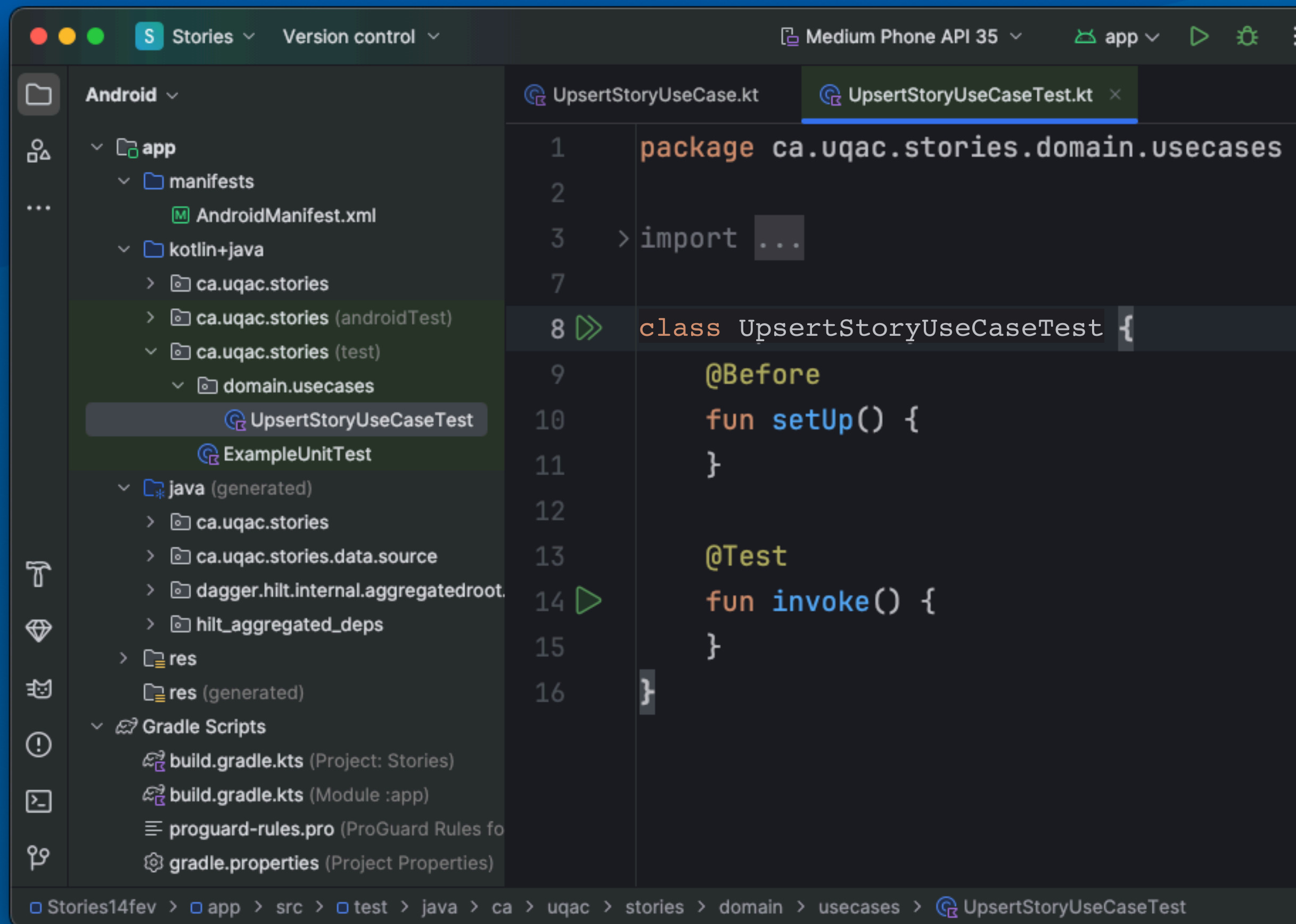
Test



● Emplacement des Tests

- `app/src/test` pour les tests unitaires
- `app/src/androidTest` pour les tests d'interface utilisateur

Structure d'un test unitaire



● Annotations JUnit

● @Before

- Méthode exécutée avant chaque test

● @Test

- Identifie une méthode de test

Initialiser des objets dans un test

```
class UpsertStoryUseCaseTest {  
  
    @Before  
    fun setUp() {  
    }  
  
    @Test  
    fun test1() {  
    }  
  
    @Test  
    fun test2() {  
    }  
  
    @Test  
    fun test3() {  
    }  
  
}
```

- Une classe de test contient **plusieurs tests** pour une même **unité à tester** (ex : un UpsertStoryUseCase)
- Ces tests doivent créer et utiliser le même objet à tester
- Mais ces objets ont souvent **des dépendances**
 - ex : StoriesDao

Initialiser des objets dans un test

```
class UpsertStoryUseCaseTest {  
    @Before  
    fun setUp() {}  
  
    @Test  
    fun test1() {  
        var upsertStoryUseCase: UpsertStoryUseCase  
    }  
  
    @Test  
    fun test2() {  
        var upsertStoryUseCase: UpsertStoryUseCase  
    }  
}
```

- **On ne veut pas recréer** ces objets dans chaque test
- duplication de code
- tests plus difficiles à maintenir
- Solution
- Définir ces objets comme variables d'instance

Nouveau problème

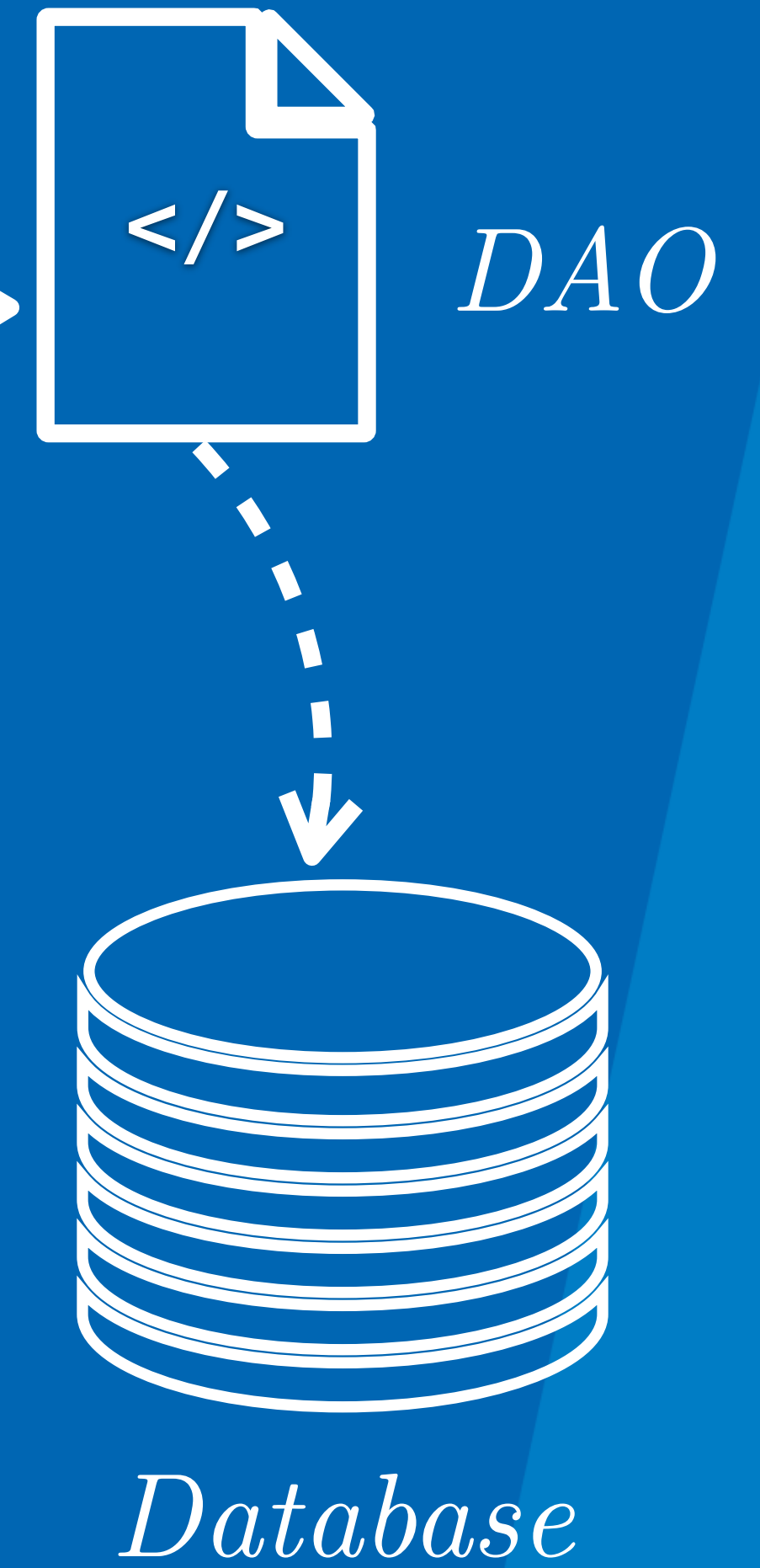
- En Kotlin, une variable doit être **initialisée immédiatement**
- Impossible :
 - les dépendances ne sont pas encore prêtes
 - on ne peut pas les passer via le constructeur du test

```
class UpsertStoryUseCaseTest {  
    var upsertStoryUseCase: UpsertStoryUseCase = ???  
  
    @Before  
    fun setUp() {}  
  
    @Test  
    fun test1() {}  
}
```



Le lateinit | On initialise l'objet plus tard

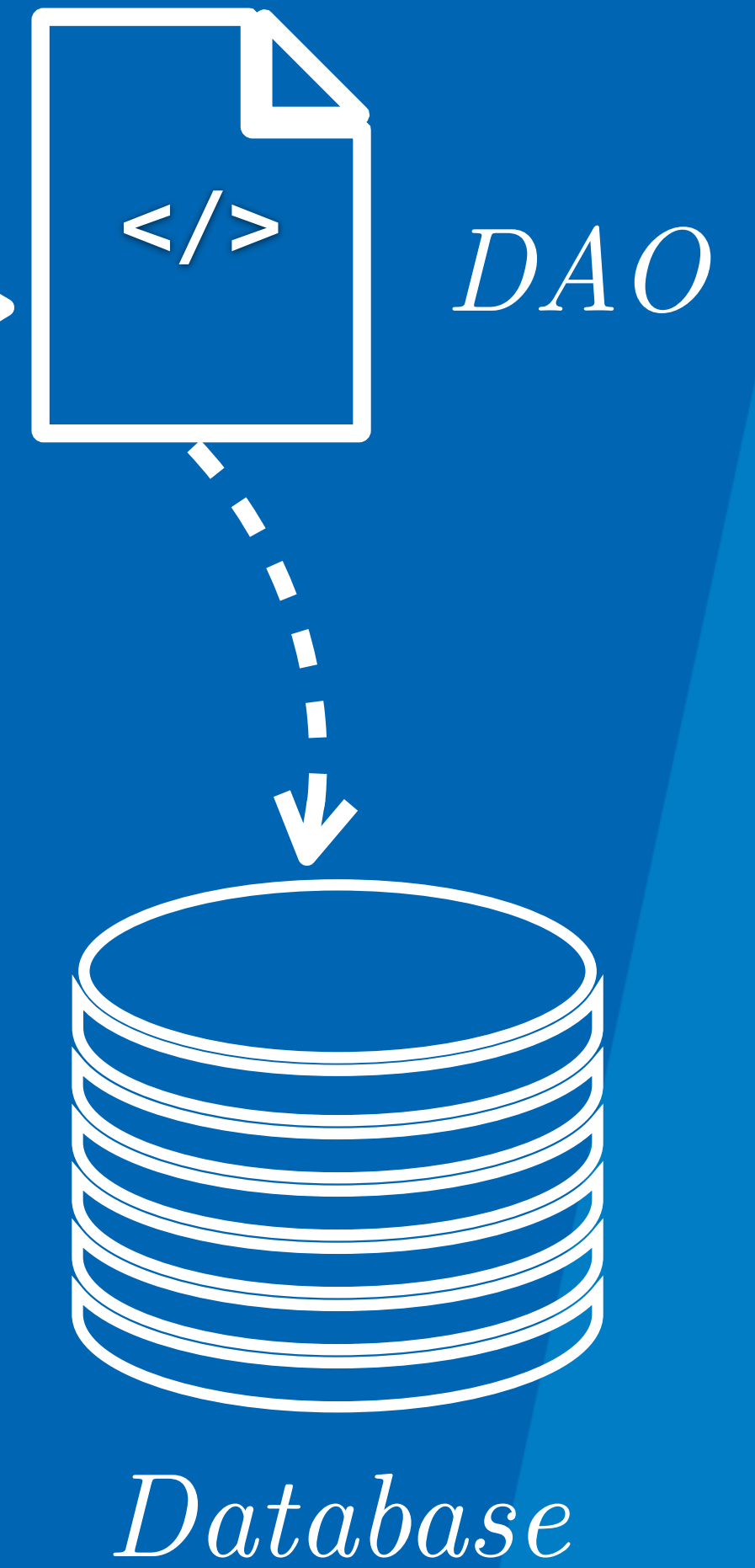
```
class UpsertStoryUseCaseTest {  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
  
    @Before  
    fun setUp() {  
        upsertStoryUseCase = UpsertStoryUseCase(??????)  
    }  
  
    @Test  
    fun invoke() {  
    }  
}
```



- **UpsertStoryUseCase** depend d'un *StoriesDao*
- Dans l'application, *StoriesDao* est implémenté par Room

Le lateinit | On initialise l'objet plus tard

```
class UpsertStoryUseCaseTest {  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
  
    @Before  
    fun setUp() {  
        upsertStoryUseCase = UpsertStoryUseCase(??????)  
    }  
  
    @Test  
    fun invoke() {  
    }  
}
```



- On instancie la "base de données" aussi
- On complete l'initialisation dans le @Before

Pas de base de données

```
class UpsertStoryUseCaseTest {
    lateinit var upsertStoryUseCase: UpsertStoryUseCase

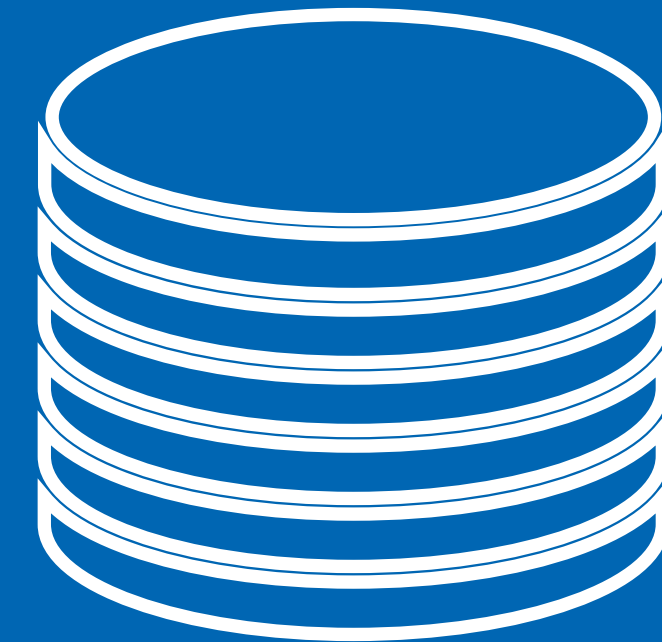
    @Before
    fun setUp() {
        upsertStoryUseCase = UpsertStoryUseCase(?????)
    }

    @Test
    fun invoke() {
    }
}
```

- Pour les tests unitaires
- **Nous n'avons pas accès** à l'implémentation réelle de la base de données
- Nous ne **voulons pas** utiliser **une vraie base** de donnée
 - Trop lourd
 - Non isolé

Solution | FakeDatabase

- Remplace la vraie base
- Simule le comportement
- Permet des tests rapides



FakeDatabase

Solution | FakeDatabase

```
package ca.uqac.stories

import ca.uqac.stories.data.source.StoriesDao

class FakeDatabase : StoriesDao {
}
```

FakeDatabase

```
package ca.uqac.stories
...

class FakeDatabase : StoriesDao {
    override fun getStories(): Flow<List<Story>> {
    }

    override fun getStory(id: Int): Story? {
    }

    override suspend fun upsertStory(story: Story) {
    }

    override suspend fun deleteStory(story: Story) {
    }
}
```


FakeDatabase

```
class FakeDatabase : StoriesDao {  
    val stories = mutableListOf<Story>()  
  
    override fun getStories(): Flow<List<Story>> {  
    }  
  
    override suspend fun getStory(id: Int): Story? {  
    }  
  
    override suspend fun upsertStory(story: Story) {  
    }  
  
    override suspend fun deleteStory(story: Story) {  
    }  
}
```

FakeDatabase

```
class FakeDatabase : StoriesDao {  
    val stories = mutableListOf<Story>()  
  
    override fun getStories(): Flow<List<Story>> = flow {  
        emit(stories)  
    }  
  
    override suspend fun getStory(id: Int): Story? {  
        return stories.find { it.id == id }  
    }  
  
    override suspend fun upsertStory(story: Story) {  
        if(stories.contains(story))  
            stories.remove(story)  
        stories.add(story)  
    }  
  
    override suspend fun deleteStory(story: Story) {  
        stories.remove(story)  
    }  
}
```


Utilisation du fake dans le test

```
class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
    @Before  
    fun setUp() {  
        upsertStoryUseCase = UpsertStoryUseCase(database)  
    }  
  
    @Test  
    fun invoke() {  
    }  
}
```

Créer les tests

Créer les test

- **Tester** : L'histoire devrait être ajoutée si les champs sont valides
 - **Arrange**
 - Créer Story
 - **Act**
 - Appeler use case
 - **Assert**
 - Vérifier résultat

Nommer les tests

- Les tests sont des fonctions annotées avec **@Test**
- Les noms de tests doivent être **descriptifs et clairs**
- En Kotlin, **nous pouvons utiliser des espaces dans les noms de fonction**

```
@Test  
fun `story should be added if fields are valid`() {  
}
```

Créer et insérer une histoire valide

Créer les données | Arrange

```
@Test
fun `story should be added if fields are valid`() {
    val story = Story(
        id = 1,
        title = "Mon Histoire",
        description = "Desc Test",
        done = false,
        priority = 1
    )
}
```

Créer et insérer une histoire valide

Appeler la méthode à tester | Action

```
@Test
fun `story should be added if fields are valid`() {
    val story = Story(
        id = 1,
        title = "Mon Histoire",
        description = "Desc Test",
        done = false,
        priority = 1
    )

    // Appeler le usecase
    upsertStoryUseCase(story)
}
}
```

invoke est une fonction suspendue

runBlocking

utiliser runBlocking pour appeler
des fonctions suspend dans les tests

```
@Test
fun `story should be added if fields are valid`() {
    runBlocking {
        val story = Story(
            id = 1,
            title = "Mon Histoire",
            description = "Desc Test",
            done = false,
            priority = 1
        )
        upsertStoryUseCase(story)
    }
}
```

bloque le thread
jusqu'à la fin
de l'exécution
- idéal pour
les tests

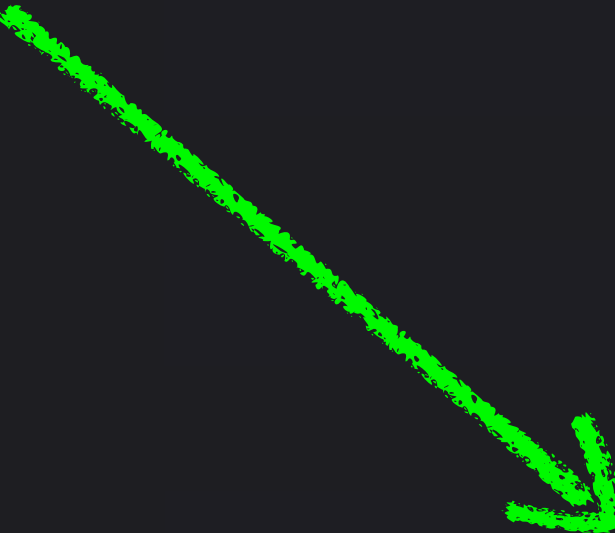
Vérifier les résultats | Assert

```
@Test
fun `story should be added if fields are valid`() {
    runBlocking {
        val story = Story(
            id = 1,
            title = "Mon Histoire",
            description = "Desc Test",
            done = false,
            priority = 1
        )

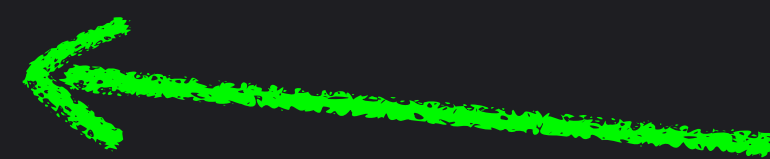
        upsertStoryUseCase(story)

        val stories = database.getStories().first()
        assertEquals(1, stories.size)
    }
}
```

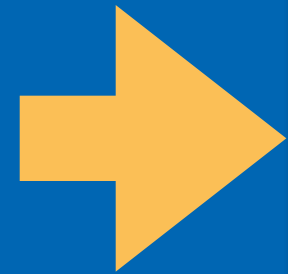
obtenir
le résultat



Vérifier que l'histoire
a été ajoutée



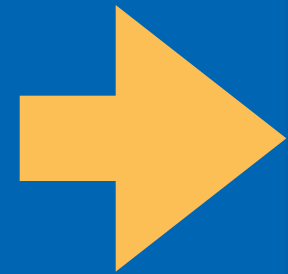
Exécuter le test



```
>> class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
    @Before  
    fun setUp() {  
        upsertStoryUseCase = UpsertStoryUseCase(database)  
    }  
  
    @Test  
    fun `story should be added if fields are valid`() {  
        runBlocking {  
            // Créer et insérer une histoire valide  
            val story = Story(  
                id = 1,  
                title = "Mon Histoire",  
                description = "Auteur Test",  
                done = false,  
                priority = 1  
            )  
  
            // Appeler le cas d'utilisation  
            upsertStoryUseCase(story)  
  
            // Vérifier que l'histoire a été ajoutée  
            val stories = database.getStories().first()  
            assertEquals(expected: 1, stories.size)  
        }  
    }  
}
```

- Cliquer sur l'icône ▶ à côté de chaque test pour l'exécuter individuellement
- Cliquer sur l'icône ▶ à côté de la classe pour exécuter tous les tests de la classe

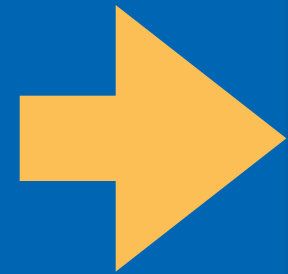
Éxecuter le test



```
>> class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
    @Before  
    fun setup() {  
        upsertStoryUseCase = UpsertStoryUseCase(database)  
    }  
  
    @Test  
    fun fields are valid() {  
        val histoire valide = UpsertStoryUseCaseRequest(  
            id = 1,  
            title = "Mon Histoire",  
            description = "Auteur Test",  
            done = false,  
            priority = 1  
        )  
  
        // Appeler le cas d'utilisation  
        upsertStoryUseCase(story)  
  
        // Vérifier que l'histoire a été ajoutée  
        val stories = database.getStories().first()  
        assertEquals(expected: 1, stories.size)  
    }  
}
```

Run 'UpsertStoryUseCaseTe...' ^⇧R
Debug 'UpsertStoryUseCaseTe...' ^⇧D
Run 'UpsertStoryUseCaseTe...' with Coverage
Modify Run Configuration...

Éxecuter le test



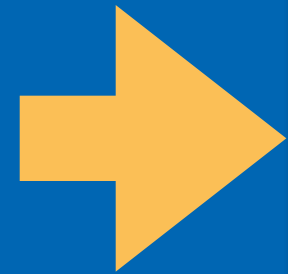
```
class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
    @Before  
    fun setUp() {  
        upsertStoryUseCase = UpsertStoryUseCase(database)  
    }  
  
    @Test  
    fun `fields are valid`() {  
        val histoire = Histoire()  
        histoire.valide()  
  
        id = 1,  
        title = "Mon Histoire",  
        description = "Auteur Test",  
        done = false,  
        priority = 1  
    }  
}
```

Run 'UpsertStoryUseCaseTe...'
Debug 'UpsertStoryUseCaseTe...'
Run 'UpsertStoryUseCaseTe...' with Coverage
Modify Run Configuration...

Run UpsertStoryUseCaseTest.story should be added if field... x

Instantiating tests... Starting...

Éxecuter le test



```
class UpsertStoryUseCaseTest {

    lateinit var upsertStoryUseCase: UpsertStoryUseCase
    val database = FakeDatabase()

    @Before
    fun setUp() {
        upsertStoryUseCase = UpsertStoryUseCase(database)
    }

    @Test
    fun `fields are valid`() {
        val histoire = Histoire(
            id = 1,
            title = "Mon Histoire",
            description = "Auteur Test",
            done = false,
            priority = 1
        )
        upsertStoryUseCase.upsert(histoire)
    }
}
```

Run 'UpsertStoryUseCaseTe...' ^⇧R
Debug 'UpsertStoryUseCaseTe...' ^⇧D
Run 'UpsertStoryUseCaseTe...' with Coverage
Modify Run Configuration...

Run UpsertStoryUseCaseTest.story should be added if field... x

Test Results 33 ms

Tests passed: 1 of 1 test – 33 ms

For more on this, please refer to https://docs.gradle.org/8.11.1/userguide/command_line_interface.html#sec:command_line_wa

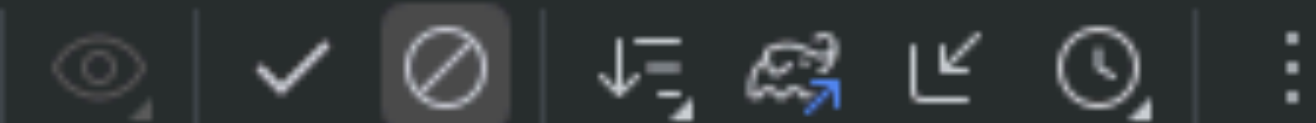
BUILD SUCCESSFUL in 25s
34 actionable tasks: 33 executed, 1 up-to-date

Build Analyzer results available
4:15:41 p.m.: Execution finished 'app:testDebugUnitTest --tests "ca.uqac.stories.domain.usecases.UpsertStoryUseCaseTest.s

Éxecuter le test

```
class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
    @Before  
    fun setUp() {  
        upsertStoryUseCase = UpsertStoryUseCase(database)  
    }  
}
```

UpsertStoryUseCaseTest.story should be added if field... x



Tests 33 ms

✓ Tests passed: 1 of 1 test – 33 ms

For more on this, please refer to https://docs.gradle.org/8.11.1/userguide/command_line_inter

BUILD SUCCESSFUL in 25s

34 actionable tasks: 33 executed, 1 up-to-date

Build Analyzer results available

4:15:41 p.m.: Execution finished ':app:testDebugUnitTest --tests "ca.uqac.stories.domain.usec

Simuler l'échec du test

```
@Test
fun `story should be added if fields are valid`() {
    runBlocking {
        val story = Story(
            id = 1,
            title = "Mon Histoire",
            description = "Desc Test",
            done = false,
            priority = 1
        )

        upsertStoryUseCase(story)

        val stories = database.getStories().first()
        assertEquals(0, stories.size)
    }
}
```


Échec du test

```
@Test
fun `story should be added if fields are valid`() {
    runBlocking {
        val story = Story(
            id = 1,
            title = "Mon Histoire",
            description = "Desc Test",

```

Tests failed: 1 of 1 test – 35 ms

> Task :app:hiltJavaCompileDebugUnitTest NO-SOURCE
> Task :app:processDebugUnitTestJavaRes UP-TO-DATE
> Task :app:transformDebugUnitTestClassesWithAsm UP-TO-DATE
> Task :app:testDebugUnitTest FAILED

Expected :0
Actual :1
[<Click to see difference>](#)

```
}
```

```
}
```

Tester les cas d'erreur

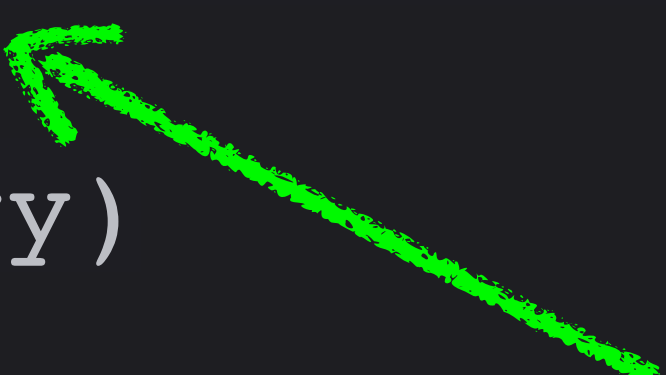
```
@Test(expected = StoryException::class)
fun `story should not be added if the title is empty`() {
}

@Test(expected = StoryException::class)
fun `story should not be added if the description is empty`() {
}
```

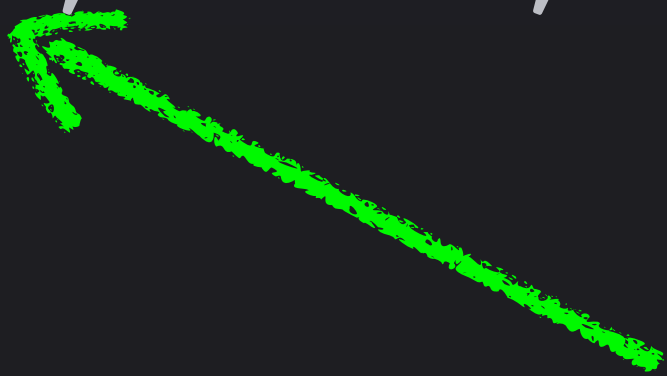
- Les erreurs font partie du comportement attendu
- On doit vérifier que **les exceptions sont bien lancées**

Tester les cas d'erreur

```
@Test(expected = StoryException::class)
fun `story should not be added if the title is empty`() {
    runBlocking {
        val story = Story(1, "", "Desc Test", false, 1)
        upsertStoryUseCase(story)
    }
}
```



```
@Test(expected = StoryException::class)
fun `story should not be added if the description is empty`() {
    runBlocking {
        val story = Story(1, "Mon Histoire", "", false, 1)
        upsertStoryUseCase(story)
    }
}
```



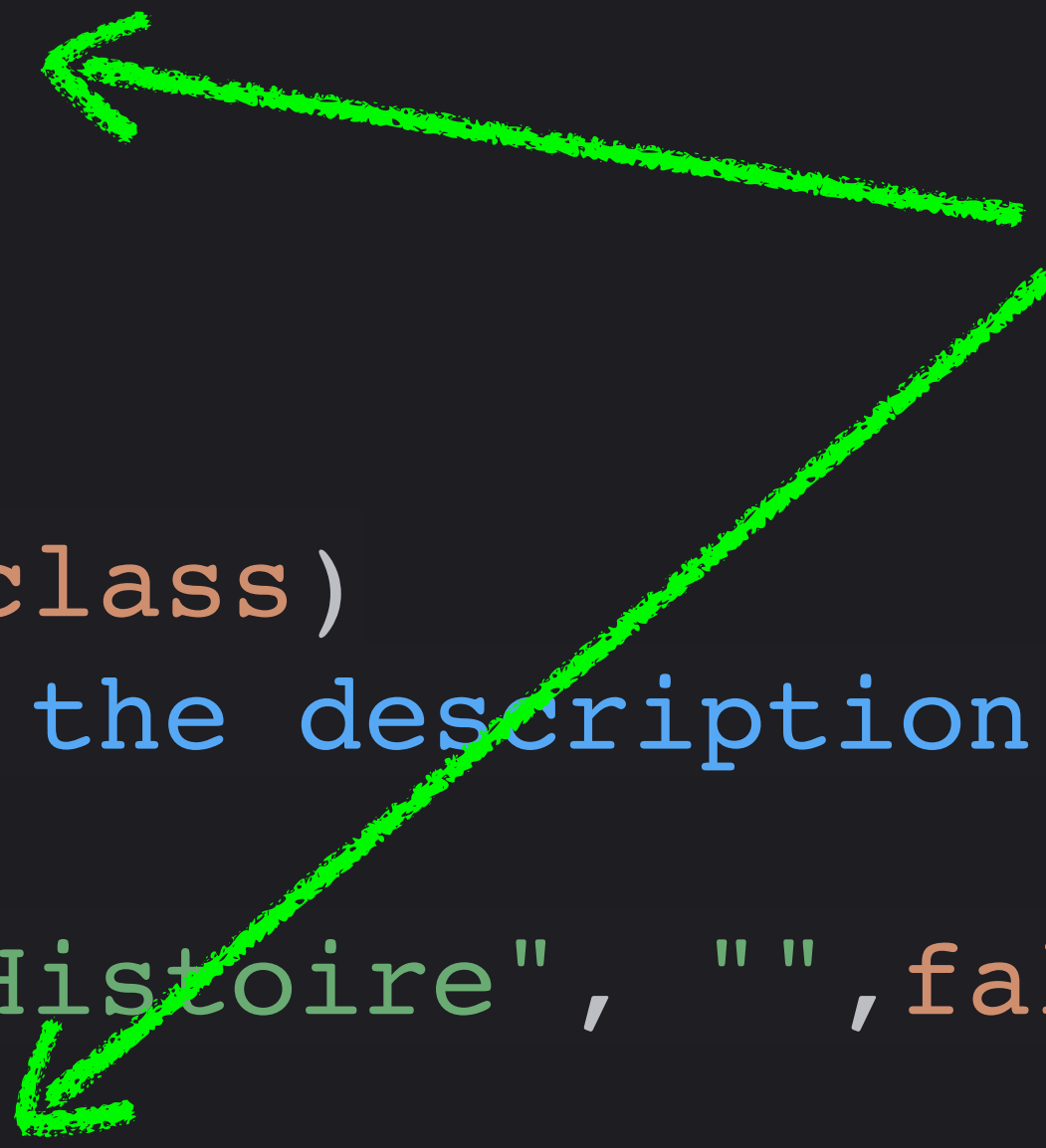
Tester les cas d'erreur

```
@Test(expected = StoryException::class)
fun `story should not be added if the title is empty`() {
    runBlocking {
        val story = Story(1, "", "Desc Test", false, 1)

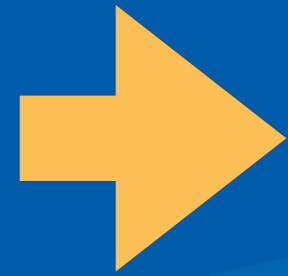
        upsertStoryUseCase(story)
    }
}
```

```
@Test(expected = StoryException::class)
fun `story should not be added if the description is empty`() {
    runBlocking {
        val story = Story(1, "Mon Histoire", "", false, 1)
        upsertStoryUseCase(story)
    }
}
```

Ces lignes devraient
lancer l'exception

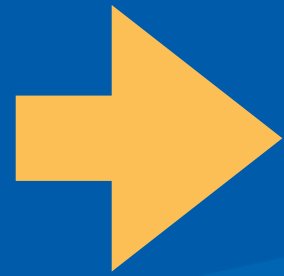


Éxecuter tous les tests



```
class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
    @Before  
    > fun setUp() {...}  
  
    @Test  
    > fun `story should be added if fields are valid`() {...}  
  
    @Test(expected = StoryException::class)  
    > fun `story should not be added if the title is empty`() {...}  
  
    @Test(expected = StoryException::class)  
    > fun `story should not be added if the description is empty`() {...}  
}
```

Éxecuter tous les tests



```
class UpsertStoryUseCaseTest {  
  
    lateinit var upsertStoryUseCase: UpsertStoryUseCase  
    val database = FakeDatabase()  
  
}
```

✓ Test Results

35 ms

✓ Tests passed: 3 of 3 tests – 35 ms

Executing tasks: [:app:testDebugUnitTest, --tests, ca.uqac.stories.domain.usecases.UpsertStoryUseCas

> Task :app:preBuild UP-TO-DATE

> Task :app:preDebugBuild UP-TO-DATE

> Task :app:checkKotlinGradlePluginConfigurationErrors SKIPPED



```
@Test(expected = StoryException::class)
```

```
fun `story should not be added if the title is empty`() {...}
```



```
@Test(expected = StoryException::class)
```

```
fun `story should not be added if the description is empty`() {...}
```

```
}
```


Checklist de tests pour applications mobiles

- **Validation des Entrées**

- Longueur (min/max)
- Caractères autorisés/interdits
- Formats (email, téléphone, etc.)

- **Valeurs Extrêmes et Limites**

- Valeurs aux limites exactes (ex: 20 caractères si max=20)
- Valeurs juste au-delà des limites (ex: 21 caractères)

- **Cas Spéciaux**

- Entrées vides ou nulles
- Zéros et valeurs négatives
- Caractères spéciaux et Unicode

Checklist de tests pour applications mobiles

- **Forcer les Erreurs**

- Provoquer chaque message d'erreur possible
- Tester les exceptions et leur gestion

- **États du Système**

- Changements de configuration (rotation d'écran)
- Mise en arrière-plan/premier plan de l'application
- Redémarrage de l'application

- **Performances**

- Grandes quantités de données
- Opérations répétitives
- Utilisation de la mémoire

Format correct

```
@Test
fun `username should be valid with correct format`() {
    // Arrange
    val validator = UserValidator()
    val validName = "JohnDoe123"

    // Act
    val result = validator.isValidUsername(validName)

    // Assert
    assertTrue(result)
}
```


Taille correct

```
@Test
fun `username should be invalid if too short`() {
    val validator = UserValidator()
    val tooShortName = "Jo"

    val result = validator.isValidUsername(tooShortName)

    assertFalse(result)
}
```

Taille correct

```
@Test
fun `should handle maximum allowed input length`() {
    val validator = UserValidator()
    val maxLengthName = "a".repeat(20) // Maximum autorisé

    val result = validator.isValidUsername(maxLengthName)

    assertTrue(result)
}
```

```
@Test
fun `should reject input exceeding maximum length`() {
    val validator = UserValidator()
    val tooLongName = "a".repeat(21) // Dépasse le maximum

    val result = validator.isValidUsername(tooLongName)

    assertFalse(result)
}
```

Vide et Nullité

```
@Test
fun `should handle empty input`() {
    val validator = UserValidator()

    val result = validator.isValidUsername("")

    assertFalse(result)
}

@Test
fun `should handle null input`() {
    val validator = UserValidator()

    val result = validator.isValidUsername(null)

    assertFalse(result)
}
```


Entrées Spéciales

```
@Test
fun `should reject input with special characters`() {
    val validator = UserValidator()
    val nameWithSpecialChars = "User@Name"

    val result = validator.isValidUsername(nameWithSpecialChars)

    assertFalse(result)
}
```

Transformations

```
@Test
fun `adding item should increase list size`() {
    // Arrange
    val shoppingCart = ShoppingCart()
    val initialSize = shoppingCart.itemCount

    // Act
    shoppingCart.addItem(Product("Téléphone", 499.99))

    // Assert
    assertEquals(initialSize + 1, shoppingCart.itemCount)
}
```

États

```
@Test
fun `removing all items should result in empty cart`() {
    val shoppingCart = ShoppingCart()
    shoppingCart.addItem(Product("Téléphone", 499.99))

    shoppingCart.clear()

    assertTrue(shoppingCart.isEmpty())
}
```


Avantages des test unitaires

- Rapides à exécuter
- Isolent précisément les problèmes
- Facilitent la refactorisation
- Servent de documentation
- Augmentent la confiance dans le code



Tests d'Interface Utilisateur (UI)

Les tests UI sont différents

- Ils testent **des interactions visuelles** et des composants interdépendants
 - Vérifient que les éléments visuels répondent correctement aux actions
- **Environnement d'exécution**
 - Ils s'exécutent sur un appareil réel ou un émulateur, pas sur la JVM locale
- **Contexte**
 - Ils nécessitent un contexte Android complet (Activity, ressources, etc.)
- **Asynchronicité**
 - Les changements d'UI sont souvent asynchrones (animations, transitions)
- **Détection**
 - On vérifie ce que l'utilisateur voit réellement à l'écran, pas des valeurs internes

Différences entre tests unitaires et tests UI

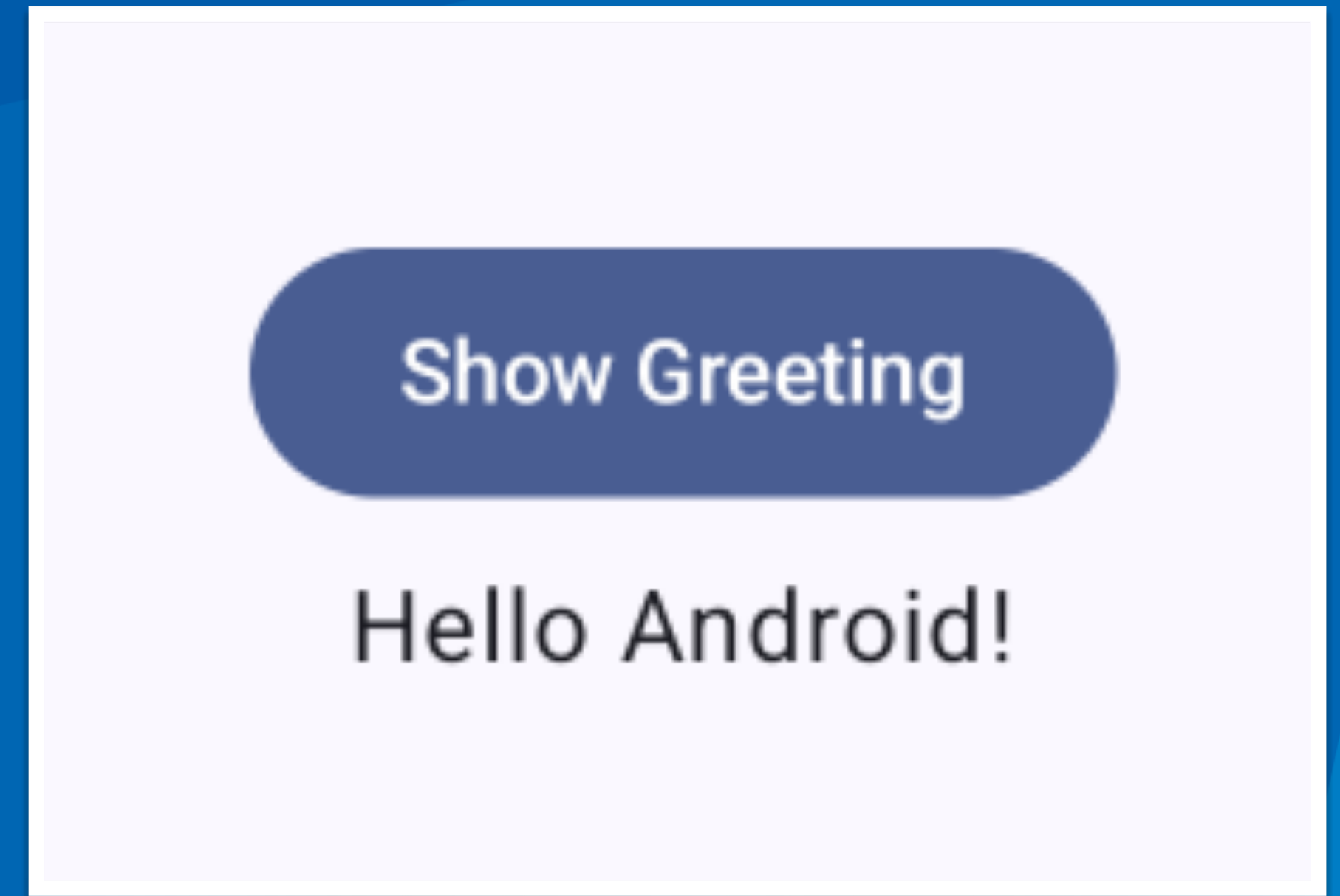
Tests Unitaires	Tests UI
Exécutés sur la JVM	Exécutés sur appareil/émulateur
Testent la logique	Testent les interactions visuelles
Indépendants du contexte Android	Dépendants du contexte Android
Utilisation d'assertions JUnit	Utilisation d'assertions spécifiques à l'UI
Rapides	Plus lents
Stables	Peuvent être instables (timing, animations)

Approche différente pour les tests UI

- Pour tester l'interface utilisateur
 - On **simule les interactions** utilisateur (clics, saisies de texte)
 - On **vérifie l'état visuel des composants**, pas leurs propriétés internes
 - On utilise des outils spécifiques comme **ComposeRule**
 - On **identifie les éléments par leur contenu visible** (texte, description) ou par **des tags de test**

L'exemple

- Une application avec
 - Un bouton
 - Un texte de salutation
 - Le texte est caché par défaut
 - Cliquer sur le bouton affiche/masque le texte



L'exemple

- Une application avec
 - Un bouton
 - Un texte de salutation
 - Le texte est caché par défaut
 - Cliquer sur le bouton affiche/masque le texte



Show Greeting

L'exemple

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    var greeting by remember{ mutableStateOf(false) }

    Column(horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize().fillMaxWidth()
    ) {
        Button(onClick = {
            greeting = !greeting
        }) {
            Text(text = "Show Greeting")
        }

        if (greeting) {
            Spacer(modifier = Modifier.height(5.dp))
            Text(text = "Hello $name!")
        }
    }
}
```

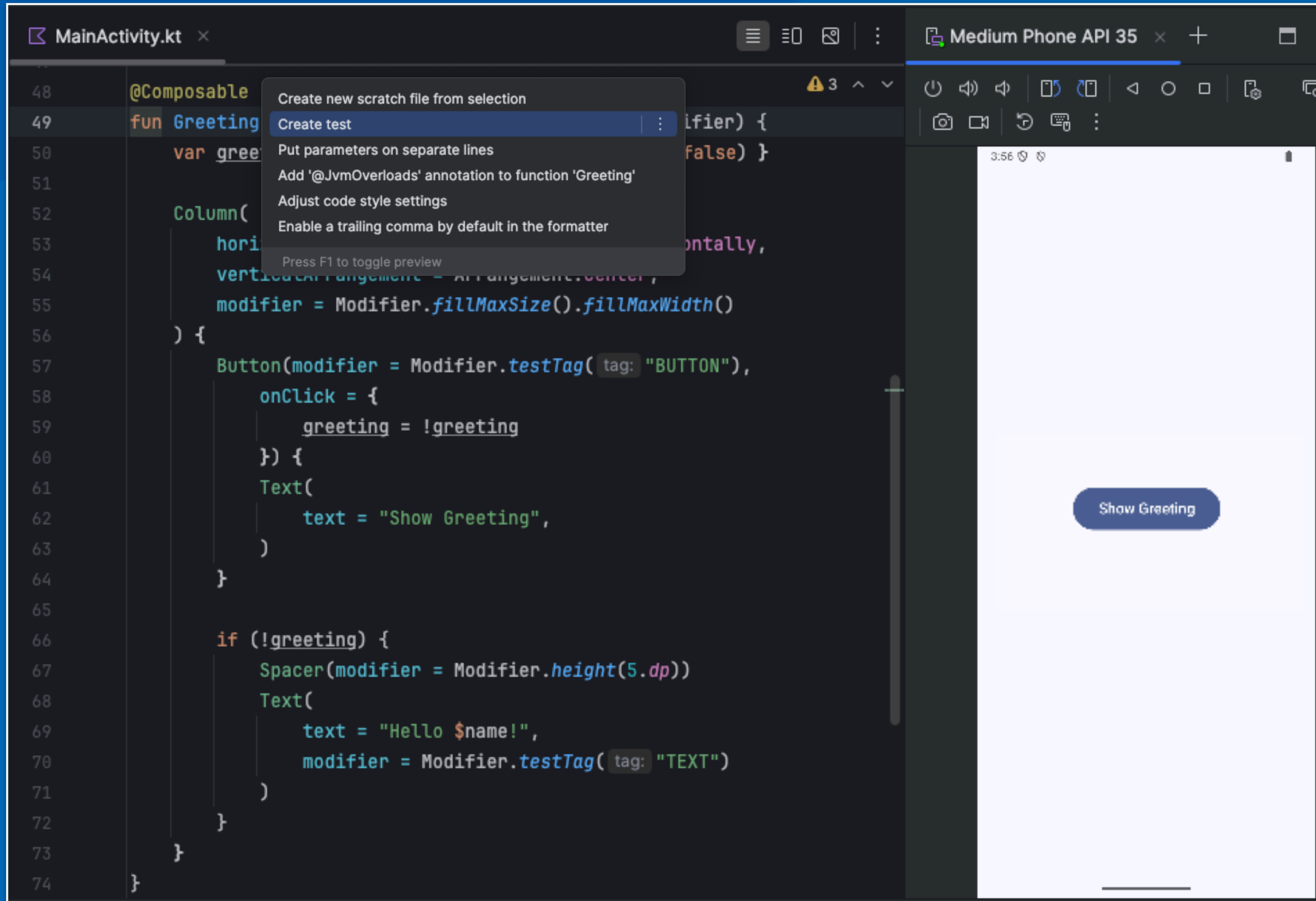
Show Greeting

Hello Android!

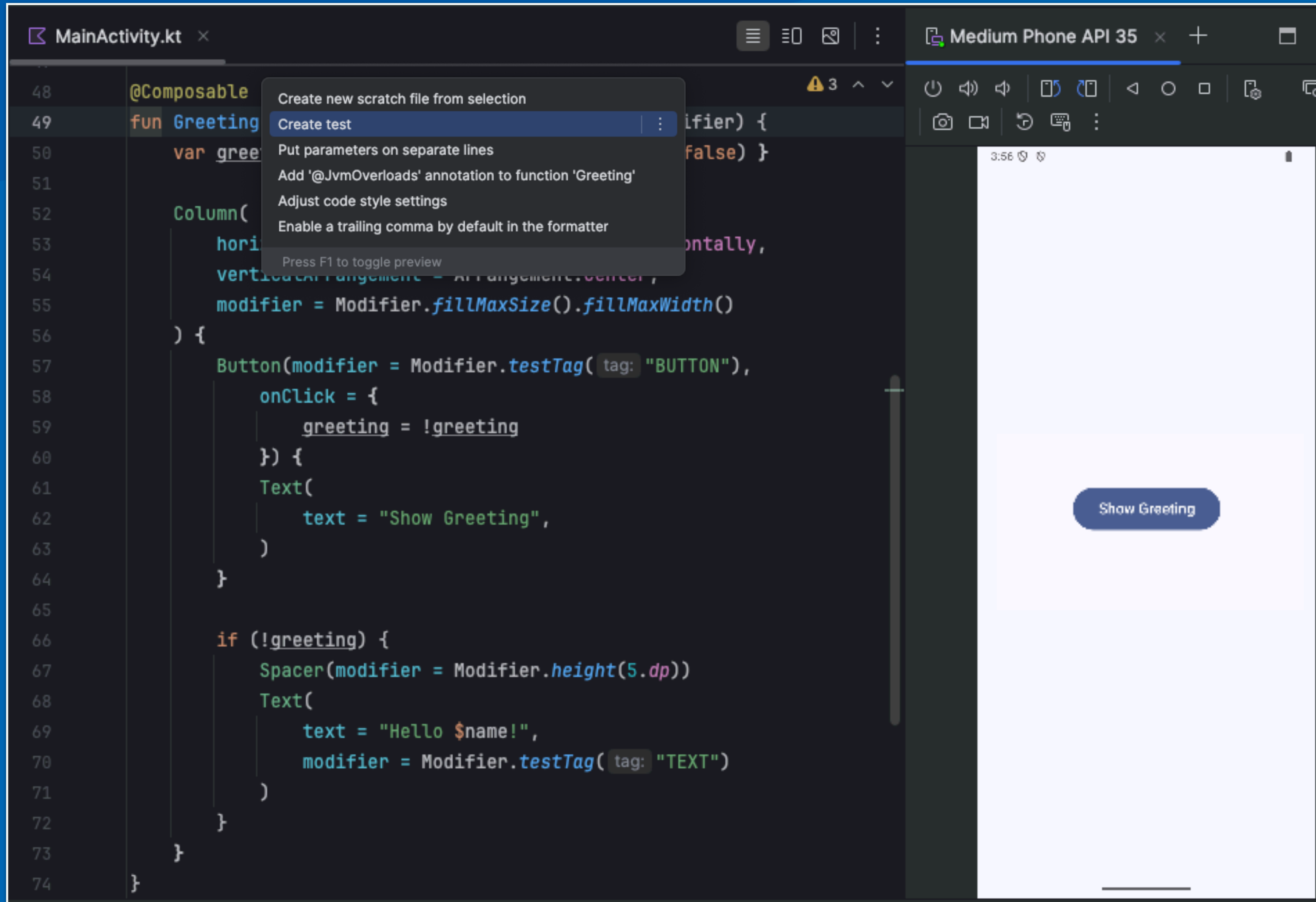
Création d'un Test UI

- **Les tests UI** sont situés dans **app/src/androidTest**
 - **Créez un test avec Alt+Enter** : Create test
 - Sélectionnez **JUnit4** comme framework (compatibilité Android)
 - Sélectionnez le package **androidTest**
 - Ce package contient des bibliothèques différents du package de tests unitaires

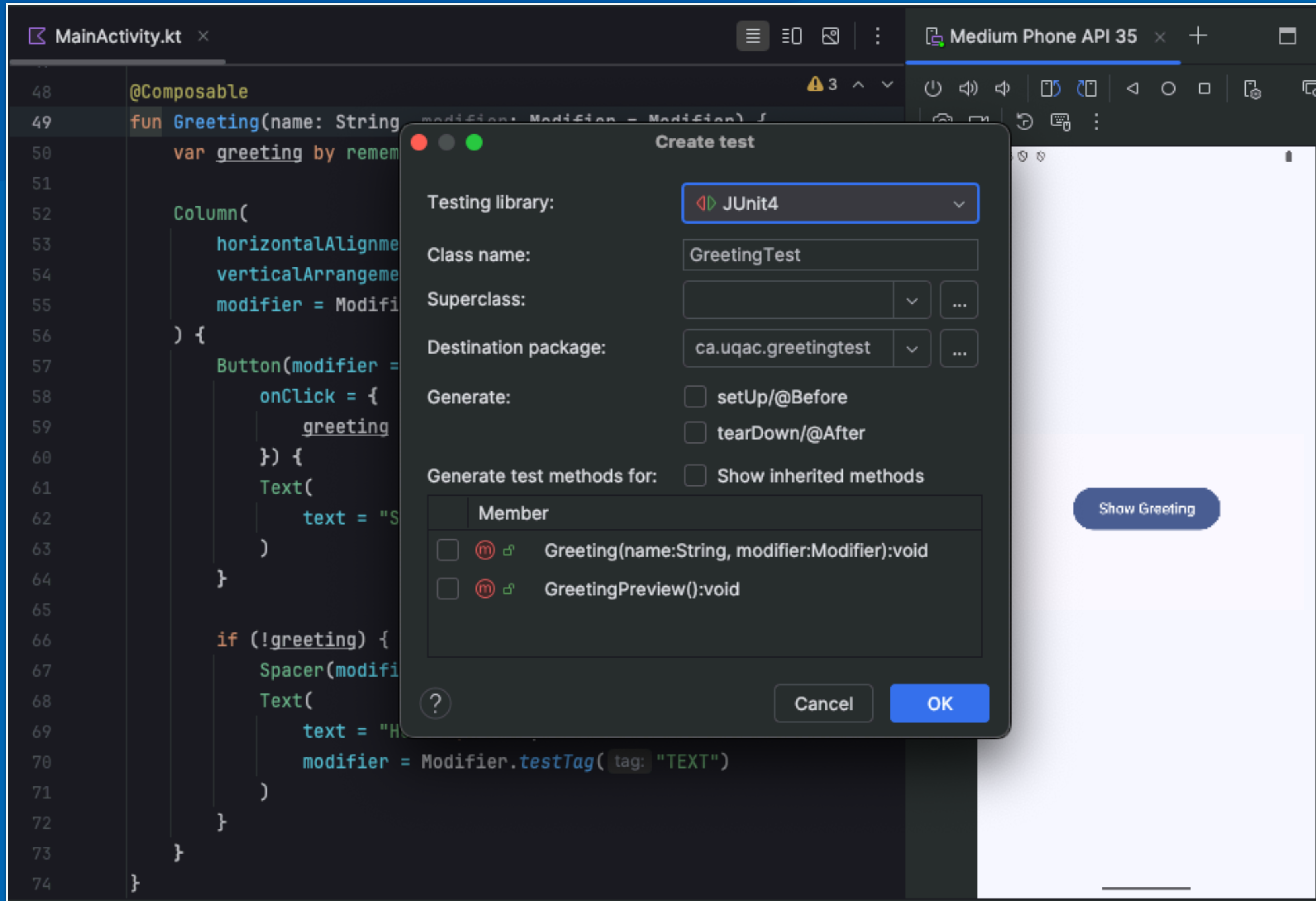
Création d'un test



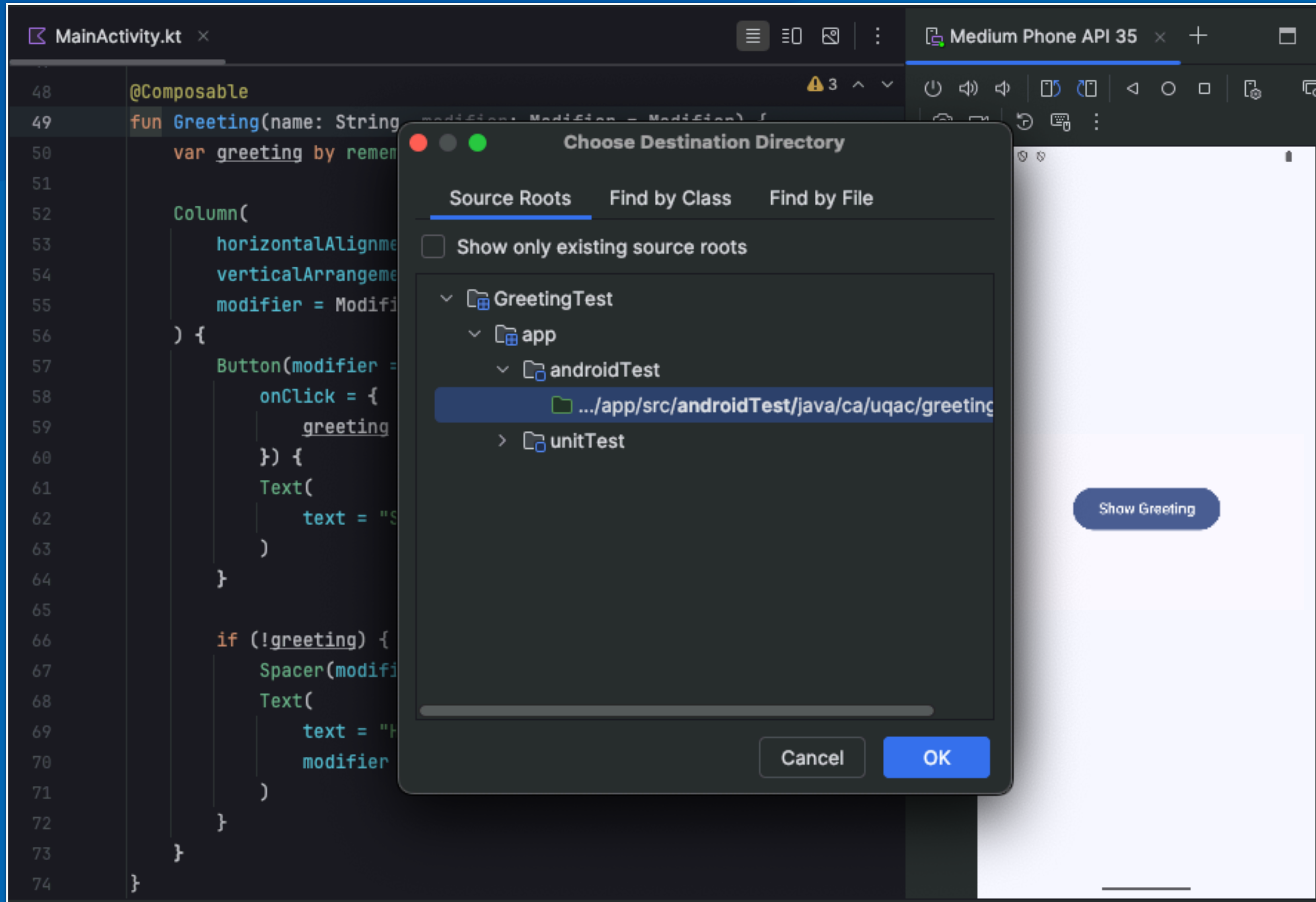
Création d'un test



Création d'un test



Création d'un test



Le test

```
package ca.uqac.greetingtest  
  
import org.junit.Assert.*  
  
class GreetingTest {  
  
}
```

Le test

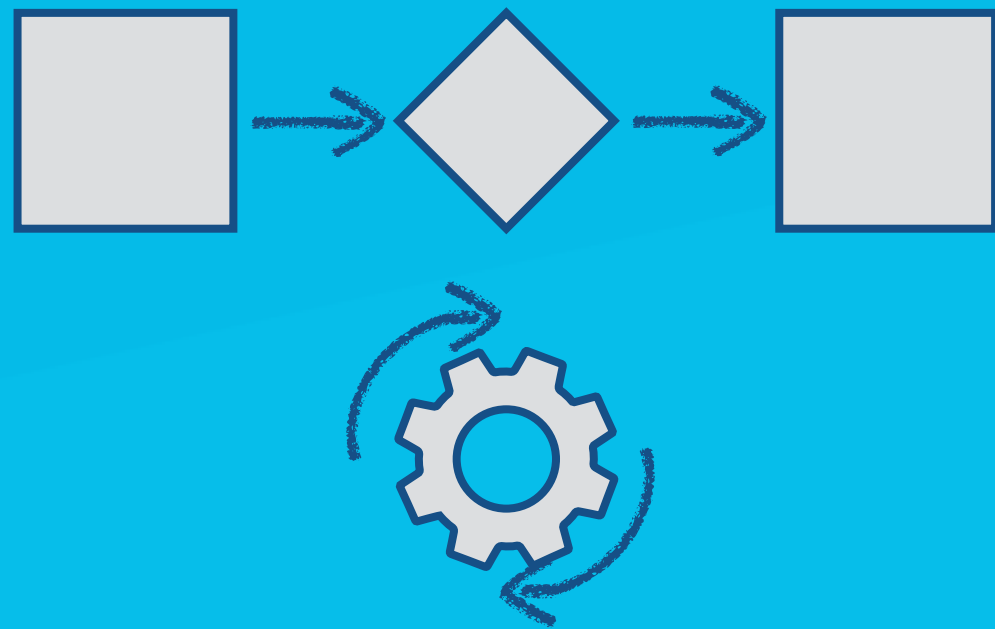
```
package ca.uqac.greetingtest  
  
import org.junit.Assert.*  
  
class GreetingTest {  
  
}  

```

On va utiliser une Compose Rule

- Nous avons besoin d'outils qui nous permettent de trouver quelque chose sur l'écran
- De faire des choses comme cliquer sur un bouton sur l'écran.

Règles JUnit



- Les **règles JUnit** permettent d'exécuter du code **avant et après** chaque test
- Permet **d'encapsuler une logique commune à plusieurs tests** pour gérer leur cycle de vie (setup/teardown) ou fournir des fonctionnalités spécifiques.

JUnit

- En Java, on utilise l'annotation **@Rule**

 **Kotlin**

- En Kotlin, on utilise **@get:Rule**
(adaptation de l'annotation Java)

ComposeRule

- **ComposeRule** est une règle spécifique *pour tester Jetpack Compose*
 - Elle initialise l'environnement Compose
 - Elle synchronise les tests avec les animations et recompositions
 - Elle fournit des API pour interagir avec l'interface

ComposeRule

- **createAndroidComposeRule<MainActivity>()**

Pour tester Compose dans le contexte d'une activité

- La règle fournit :

- **setContent { }** : Pour définir le contenu à tester

- API de sélection d'éléments (**onNodeWithTag**, etc.)

- API d'actions (**performClick**, etc.)

- API d'assertions (**assertExists**, etc.)

Structure d'un test compose

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
}
```

Il doit être initialisé avant l'exécution de tout test.

Structure d'un test compose

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
        composeRule.activity.setContent {  
  
        }  
    }  
}
```

MainActivity

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            GreetingTestTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }  
    }  
}
```


Structure d'un test compose

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
        composeRule.activity.setContent {  
            GreetingTestTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }  
    }  
}
```

Maintenant, nous pouvons
créer un test

Test UI

- Je souhaite tester
que le message d'accueil (*greeting*)
est masqué au lancement de l'application
 - « *greeting should be hidden* »
- Nous utiliserons notre « Rule »
pour accéder aux éléments de l'interface



Show Greeting

Test UI

- Je souhaite tester que le message d'accueil (*greeting*) est masqué au lancement de l'application
- « *greeting should be hidden* »
- Nous utiliserons notre « Rule » pour accéder aux éléments de l'interface



Show Greeting

```
@Test
fun `greeting should be hidden`() {
    composeRule.onNode
}

Ⓜ onNode(matcher: SemanticsMatcher, useUnmergedTree:
Ⓜ onAllNodes SemanticsNodeInteractionCollection
ⓕ onNodeWithTag(testTa... SemanticsNodeInteraction
ⓕ onNodeWithText(text:... SemanticsNodeInteraction
ⓕ onNodeWithContentDescription SemanticsNodeInt...
ⓕ onAllNodesWithContentDescription SemanticsNod...
ⓕ onAllNodesWithTag SemanticsNodeInteractionCol...
ⓕ onAllNodesWithText SemanticsNodeInteractionCo...
```

Press ↵ to insert, → to replace

Sélecteurs disponibles

- ***onNodeWithTag()***

- trouve un élément avec un testTag spécifique
- on applique des balises (*tags*) au composant

- ***onNodeWithText()***

- trouve un élément contenant un texte spécifique

- ***onNodeWithContentDescription()***

- trouve un élément avec une description spécifique

- ***onNodeWithResourceId()***

- trouve un élément avec un ID de ressource spécifique

Identifier les éléments UI avec TestTag

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    var greeting by remember{ mutableStateOf(false) }

    Column(horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
        modifier = Modifier.fillMaxSize().fillMaxWidth()
    ) {
        Button(onClick = {
            greeting = !greeting
        }) {
            Text(text = "Show Greeting")
        }

        if (greeting) {
            Spacer(modifier = Modifier.height(5.dp))
            Text(text = "Hello $name!")
        }
    }
}
```

Show Greeting

Hello Android!

Identifier les éléments UI avec TestTag

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    var greeting by remember{ mutableStateOf(false) }

    Column(horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
        modifier = Modifier.fillMaxSize().fillMaxWidth()
    ) {
        Button(modifier = Modifier.testTag("BUTTON"),
            onClick = {
                greeting = !greeting
            }) {
            Text(text = "Show Greeting")
        }

        if (greeting) {
            Spacer(modifier = Modifier.height(5.dp))
            Text(text = "Hello $name!",
                modifier = Modifier.testTag("TEXT"))
        }
    }
}
```


Assertions disponibles

- ***assertExists()***

- vérifie que l'élément existe dans l'arbre sémantique

- ***assertDoesNotExist()***

- vérifie que l'élément n'existe pas

- ***assertIsDisplayed()***

- vérifie que l'élément est affiché à l'écran

- ***assertIsNotDisplayed()***

- vérifie que l'élément n'est pas affiché

- ***assertIsEnabled()* / *assertIsNotEnabled()***

- vérifie l'état d'activation

Tester l'état initial

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
    }  
}
```

Tester l'état initial

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
    }  
  
    @Test  
    fun greetingShouldBeHidden() {  
    }  
}
```


Tester l'état initial

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
    }  
  
    @Test  
    fun greetingShouldBeHidden() {  
        composeRule  
            .onNodeWithTag( "TEXT" )  Accéder au texte  
    }  
}
```

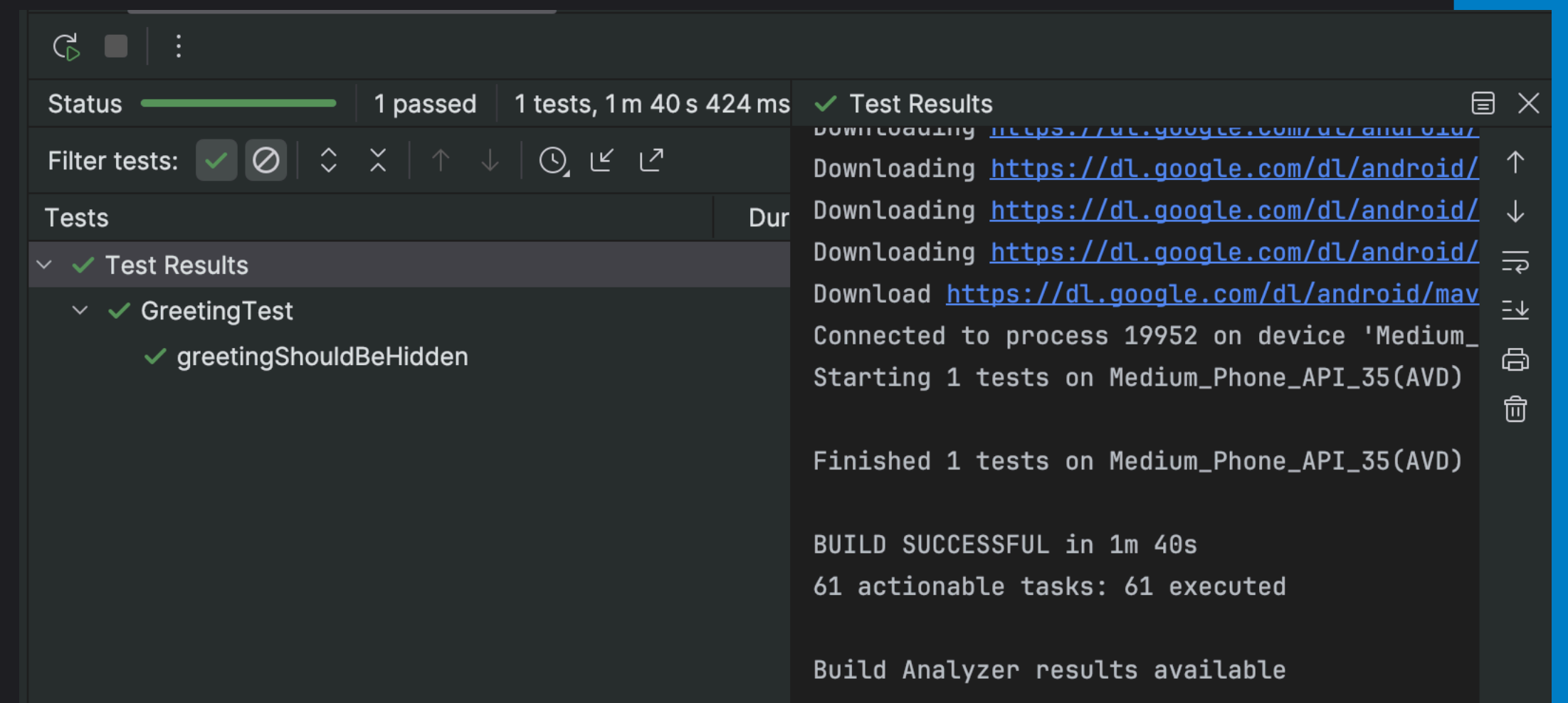
Tester l'état initial

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
    }  
  
    @Test  
    fun greetingShouldBeHidden() {  
        composeRule  
            .onNodeWithTag( "TEXT" )  
            .assertDoesNotExist()  
    }  
}
```

Vérifiez qu'il n'existe pas dans l'interface

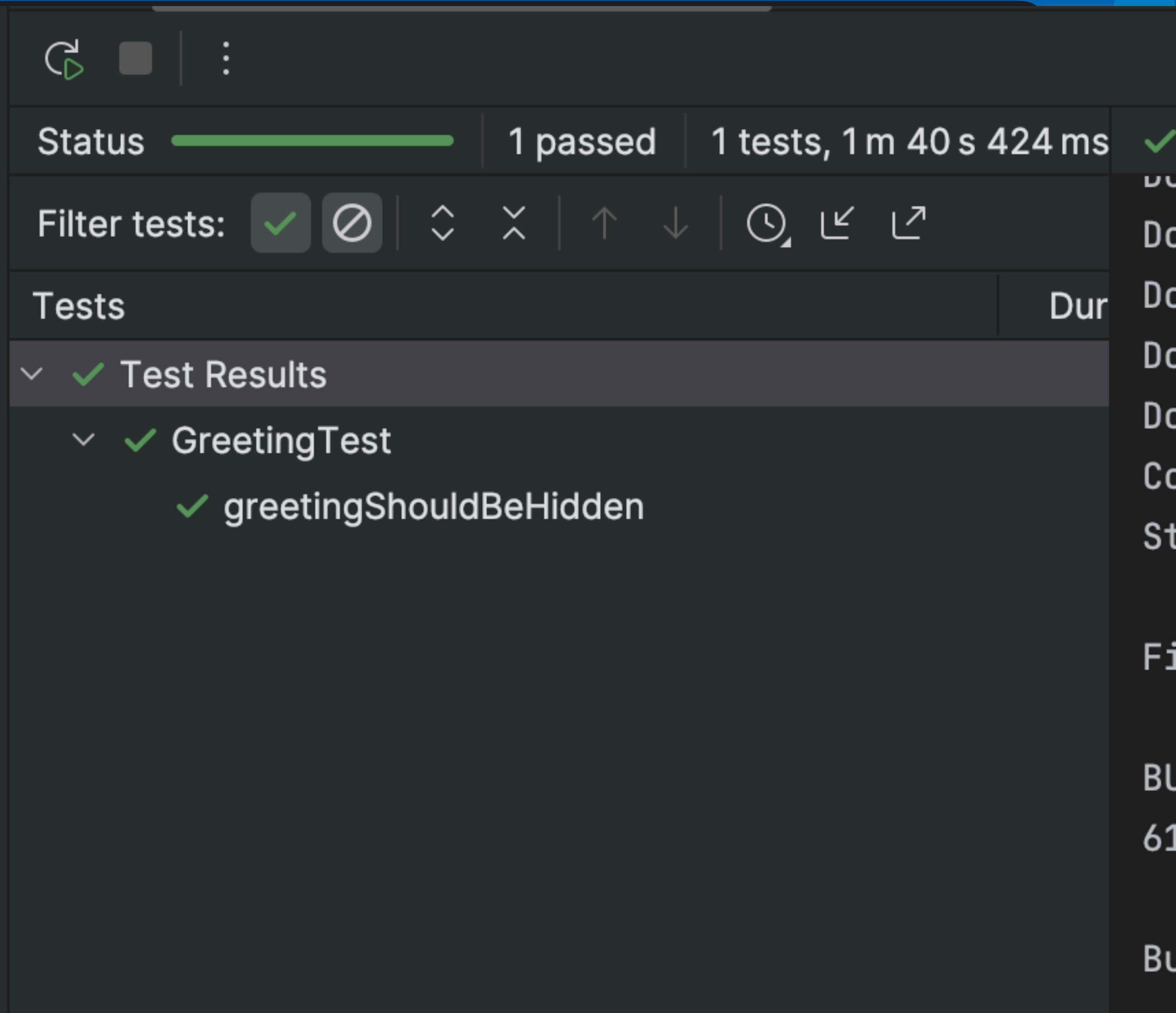
Tester l'état initial

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
    }  
  
    @Test  
    fun greetingShouldBeHidden() {  
        composeRule  
            .onNodeWithTag( "TEXT" )  
            .assertDoesNotExist()  
    }  
}
```



Tester l'état initial

```
class GreetingTest {  
    @get:Rule  
    val composeRule = createAndroidComposeRule<MainActivity>()  
  
    @Before  
    fun setup() {  
    }  
  
    @Test  
    fun greetingShouldBeHidden() {  
        composeRule  
            .onNodeWithTag("TEXT")  
            .assertDoesNotExist()  
    }  
}
```



The screenshot shows the Android Studio interface with the test results panel open. The top bar indicates the test status: "Status" with a green progress bar, "1 passed", and "1 tests, 1 m 40 s 424 ms". Below this, the "Filter tests:" section shows a green checkmark icon and a greyed-out "X" icon, along with various filter icons. The "Tests" section lists the test results:

- Test Results
 - GreetingTest
 - greetingShouldBeHidden

Interagir avec l'interface

```
@Test
fun greetingShouldAppearWhenButtonIsClicked() {
    composeRule
        .onNodeWithTag( "BUTTON" )
        .performClick() Cliquez sur le bouton avec la Rule
}
```

Interagir avec l'interface

```
@Test
fun greetingShouldAppearWhenButtonIsClicked() {
    composeRule
        .onNodeWithTag( "BUTTON" )
        .performClick()

    composeRule
        .onNodeWithTag( "TEXT" )
        .assertExists() Vérifiez qu'il existe dans l'interface
}
```


Tests d'interactions multiples

```
@Test
fun greetingShouldAppearWhenButtonIsClickedTwice() {
    composeRule
        .onNodeWithTag( "BUTTON" )
        .performClick()

    composeRule
        .onNodeWithTag( "BUTTON" )
        .performClick()

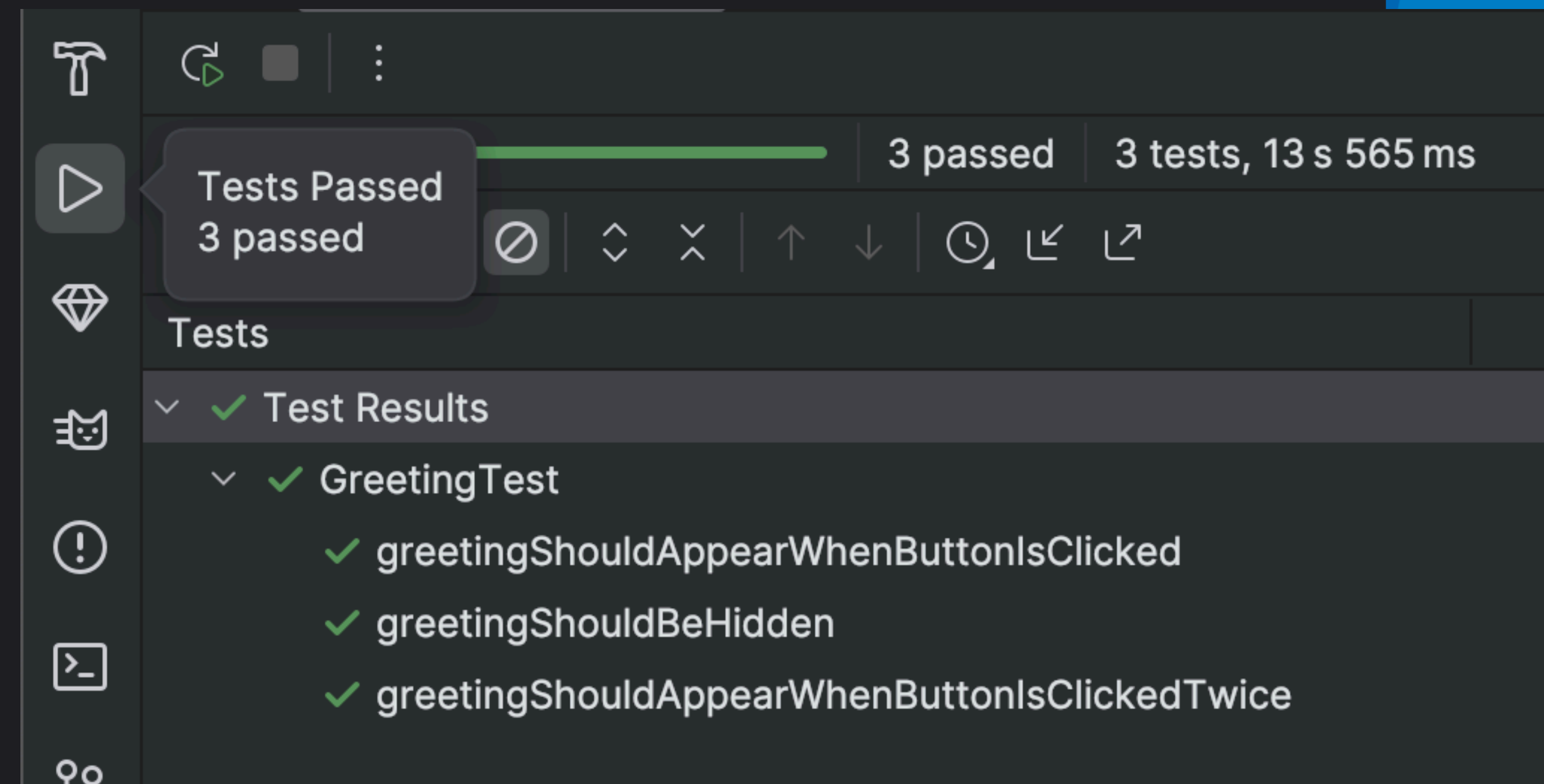
    composeRule
        .onNodeWithTag( "TEXT" )
        .assertDoesNotExist()
}
```

Tests d'interactions multiples

```
@Test
fun greetingShouldAppearWhenButtonIsClickedTwice() {
    composeRule
        .onNodeWithTag( "BUTTON" )
        .performClick()

    composeRule
        .onNodeWithTag( "BUTTON" )
        .performClick()

    composeRule
        .onNodeWithTag( "TEXT" )
        .assertDoesNotExist()
}
```



Actions disponibles

- ***performClick()***
 - simule un clic
- ***performTextInput()***
 - saisit du texte
- ***performScrollTo()***
 - fait défiler jusqu'à l'élément
- ***performTouchInput { ... }***
 - effectue des gestes complexes

Lecture et Références

- <https://developer.android.com/training/testing/fundamentals>
- <https://developer.android.com/training/testing/local-tests>
- Sommerville, Ian. Engineering software products. Vol. 355. London: Pearson, 2020.